FACULTY OF INFORMATION SCIENCE

UNIVERSITI TEKNOLOGI MARA SELANGOR BRANCH, CAMPUS PUNCAK PERDANA (UiTM)

BACHELOR OF INFORMATION SCIENCE INFORMATION SYSTEM MANAGEMENT **(CDIM262)**

COURSE:

ADVANCED WEB DEVELOPMENT AND CONTENT MANAGEMENT **(IMS566)**

GROUP PROJECT:

WEB APPLICATION WITH DATABASE INTERACTION PROJECT **(RENTIFY)**

PREPARED BY:

MUHAMAD ZULFADLI BIN JUMAAT	2025157497
MUHAMMAD HAZIQ BIN ARIFF ‘ARIFFIN	2025198985
MUHAMMAD AMIRUL RASYID BIN ROSLI	2025160577
SAFA MUHAMMAD RAZIQ BIN ABD RAFAR	2025516833

GROUP:

CDIM2624A

PREPARED FOR:

DR. MUHAMMAD ASYRAF BIN WAHI ANUAR

SUBMISSION DATE:

2nd FEBRUARY 2026

GROUP PROJECT:

WEB APPLICATION WITH DATABASE INTERACTION PROJECT **(RENTIFY)**

BY:

MUHAMAD ZULFADLI BIN JUMAAT	2025157497
MUHAMMAD HAZIQ BIN ARIFF ‘ARIFFIN	2025198985
MUHAMMAD AMIRUL RASYID BIN ROSLI	2025160577
SAFA MUHAMMAD RAZIQ BIN ABD RAFAR	2025516833

GROUP:

CDIM2624A

PREPARED FOR:

DR. MUHAMMAD ASYRAF BIN WAHI ANUAR

UNIVERSITI TEKNOLOGI MARA (UiTM)

KAMPUS PUNCAK PERDANA, SHAH ALAM

INFORMATION SCIENCE STUDIES

ACKNOWLEDGMENT

First and foremost, all praises and thanks are due to **Allah (SWT)**, the Most Gracious and the Most Merciful. We are eternally grateful for His blessings, which gave us the strength, good health, patience, and determination required to complete this group assignment and navigate the challenges we faced throughout the development of this project. Without His will and guidance, this achievement would not have been possible.

We would like to express our deepest and most sincere gratitude to our lecturer, **DR. MUHAMMAD ASYRAF BIN WAHI ANUAR**, for his continuous guidance, constructive feedback, and valuable insights. His expertise and dedication have been instrumental in shaping the direction and quality of this study. We truly appreciate his patience in reviewing our progress and his willingness to share his knowledge, which has significantly improved our understanding of the subject matter and refined the outcome of our project.

We also wish to extend our appreciation to **Universiti Teknologi MARA (UiTM)** for providing us with a conducive learning environment and excellent facilities. The accessibility to various resources, including the library and computer laboratories, played a vital role in facilitating our research and the technical development of this project. These facilities greatly contributed to the smooth execution of our work.

Our sincere appreciation is also extended to all the respondents and participants who willingly took part in our questionnaires and usability testing sessions. We are grateful for the time and effort they contributed. Their honest feedback, detailed opinions, and cooperation provided us with meaningful data that was crucial in guiding the design decisions and necessary improvements for our project.

Furthermore, we would like to thank our parents for their unwavering prayers and support, which served as our pillar of strength. A special thanks also goes to our friends and peers who offered moral support, encouragement, and assistance, whether directly or indirectly, during the ups and downs of this journey. The exchange of ideas and mutual motivation helped us stay focused on our goals.

Finally, this project is the result of the hard work and commitment of our team members, who worked tirelessly together to bring this vision to life.

TABLE OF CONTENTS

1.0 INTRODUCTION

Figure 1: Company Logo

1.1 Project Overview

Rentify is a high-quality, web-based Car Rental Management System that is created to transform the traditional paperwork into a centralized and high-performance web-based application to modernize the vehicle rental business. Rentify project is based on the CakePHP 5.x framework and uses a contemporary glassmorphism design style to provide an administrator with a Command Center experience and a customer with an easy-to-use and mobile-friendly interface. The system enables a complete shift in the manual form-processing to a highly advanced digital ecosystem by incorporating secure authentication, full CRUD (Create, Read, Update, Delete) features, and automated data export features.

1.2 Purpose and Objectives

Rentify is mainly aimed at removing manual errors and streamlining fleet management by providing real-time data visibility and automated processes. **The targeted goals of this project are:**

- **Booking Lifecycle Automation**: To automate the process of selecting a car, issuing digital invoices and automated status reports.
- **Data Integrity & Conflict Prevention**: To introduce strict backend validation to avoid overlapping bookings and database consistency.
- **Operational Intelligence:** To deliver financial KPIs and fleet performance analytics to administrators through an interactive dashboard.
- **Safety & Maintenance Monitoring:** To incorporate a vehicle health tracking system that guarantees long life of the fleet and improves the safety of customers.
- **Secure Access Control:** To ensure the safety of sensitive user information and financial records with the help of a strong Role-Based Access Control (RBAC).

1.3 Target Audience

Rentify is designed to cater to two different groups of users with different functional needs:

1.3.1 Administrators (Rental Business Owners and Staff)

The administrative component is the business operational core. **Responsibilities include:**

- Administering vehicle inventory, types, and maintenance.
- The Intelligence Dashboard for monitoring business health.
- Monitoring sources of revenue and handling customer invoices/payments.

1.3.2 Customers (Car Renters)

The interface that is exposed to the customers is concerned with usability and efficiency. **Features include:**

- Active navigation and sifting of cars on offer.
- Safe reservation control and availability in real-time.
- Availability of personal rental history, online receipts and feedback mechanisms.

2.0 GIT HUB REPOSITORY LINK

https://github.com/haziqariff703/Rentify.git

3.0 ENTITY-RELATIONSHIP DIAGRAM (ERD)

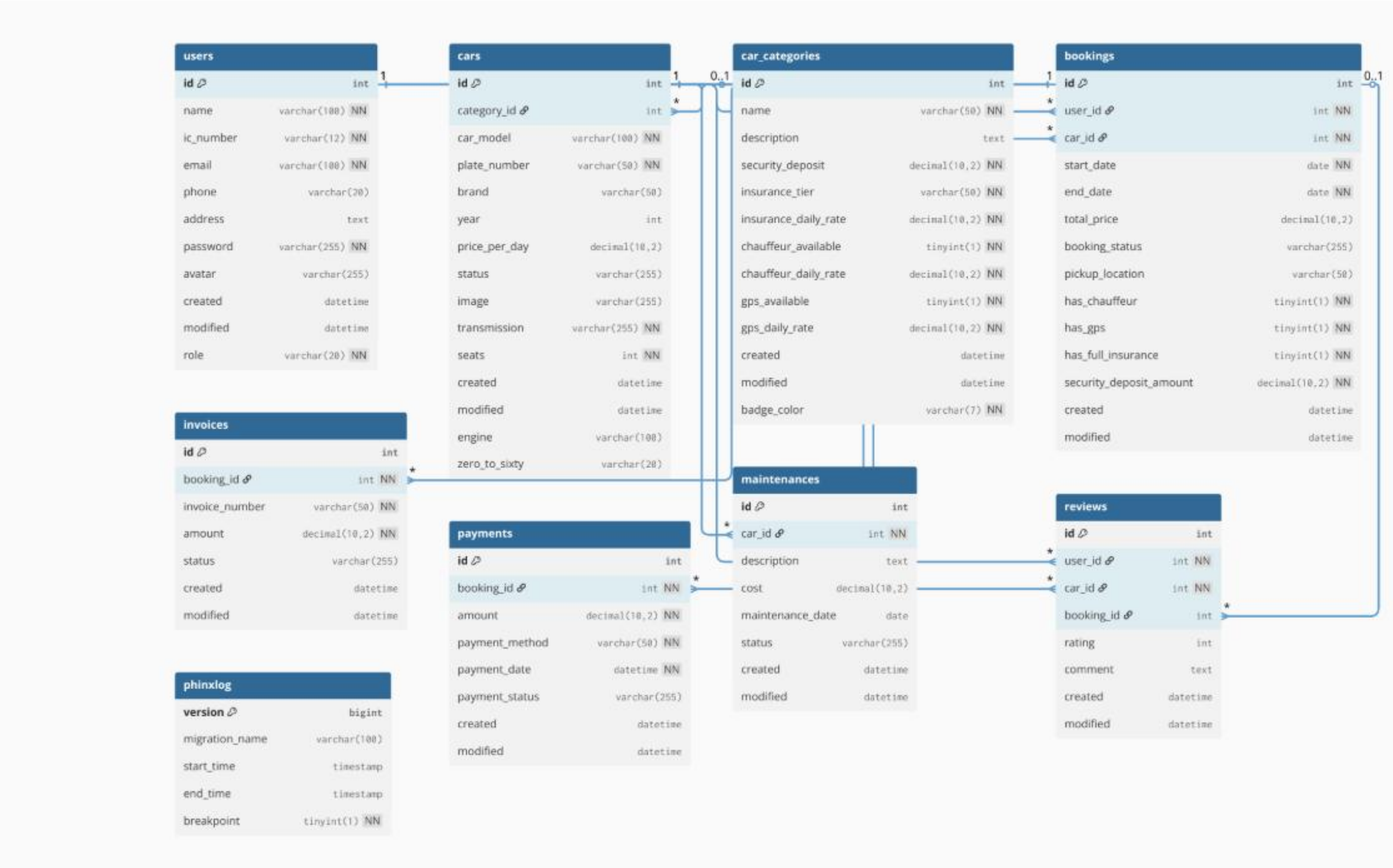


Figure 2: ERD

3.1 Entity Descriptions

Entity	Purpose
Users Table	Stores customer information like name, email, phone number, address, and password. Each user gets a unique ID.
Cars Table	Contains all the vehicles available for rent. It records the car model, license plate, brand, year, price per day, and whether the car is available or currently rented out.
Car_Categories Table	Groups cars by type (economy, luxury, SUV, etc.). Each category has its own pricing for insurance, GPS, and chauffeur services.

Bookings Table	Records each rental. It shows which user rented which car, when they picked it up, when they returned it, and the total price they paid.
Invoices Table	Create bills for each rental with invoice number, total amount due, and payment status (paid/unpaid).
Payments Table	Tracks all payments. It records how much was paid, when, payment method, and which invoice/booking it was for.
Maintenances Table	Keeps a record of when cars are serviced, what work was done, and how much it cost.
Reviews Table	Customers can rate and review cars after renting them.
Phinxlog Table	A technical table that tracks database updates and changes

Table 1: ERD Descriptions

4.0 SYSTEM REQUIREMENTS

The implementation of the Rentify Car Rental Management System is based on a particular arrangement of the hardware and software components. In this section, the technical environment to be created to facilitate the robust backend logic of the system, secure data management and user-responsive interface are outlined. These technologies were chosen on the basis of the requirement to follow the Model-View-Controller (MVC) architecture, strict typing, and the ability to implement complex business logic (automated invoicing and prevention of overlaps) smoothly.

4.1 Server & Environment Specifications

The system is configured to work in a standardized environment to provide consistency in its development and viability in deployment. This consistency played a significant role in ensuring that the team reduced the problems of it works on my machine as well as ensuring that the team followed the academic requirements given to the IMS566 course.

Figure 3: Laragon

- **OS / Local Environment:** The system was written in Windows 10/11 as the main OS with the help of Laragon development suite. Laragon was chosen due to its remote and portable developmental environment that includes the required Apache, MySQL, and PHP binaries without modifying global system settings.

Figure 4: Apache HTTP Server

- **Web Server (Apache HTTP Server):** Apache is the web server back-end. It is necessary to support the URL rewriting (mod_rewrite) that is needed by the CakePHP framework. This enables the application to delegate pretty URLs (/bookings/add) to the appropriate controller actions and hide the underlying file structure to the end-user.

Figure 5: PHP 8.1

- **Server-Side Code (PHP 8.1+):** The application code is coded in PHP 8.1, which is the strict minimum as imposed by the course rubric. This introduction of strong typing, arguments and property promotion of constructors makes a huge improvement in the readability and maintainability of the codebase. It makes sure that the complicated calculation logic, the time of rental and the overall pricing, is performed with accuracy and speed.

Figure 6: MySQL

- **Database Management System (MySQL 8.0):** MySQL is used as the persistent storage layer. The relational database management system (RDBMS) is a critical decision in Rentify since it ensures referential integrity between the 8 core entities (Users, Cars, Bookings, Invoices, etc.). Foreign keys are used to ensure that you do not have orphaned records. For example, a Booking cannot exist without a User and a Car.

Figure 7: Chrome

- **Client-Side Environment (Google Chrome):** Google Chrome was used as the default browser to test all the frontend as per the project requirements. This was used to provide consistency in the way the CSS styling and JavaScript execution is performed especially when it comes to testing responsive elements and date-picker interfaces.

4.2 Software Architecture & Dependencies

Figure 8: CakePHP

Figure 9: Composer

Rentify is developed in **CakePHP 5.2** a recent PHP framework that follows the Model-View-Controller (MVC) pattern. In this architecture, the concerns of the application are separated: The Model is responsible of data integrity and business rules, the View deals with the user interface, and the Controller deals with user input.

A few specific libraries and plugins were included through Composer in order to expand the functionality of the core of the framework:

A. **Security and Access control (Authentication Plugin)**
The official cakephp/authentication plugin is used to enforce security. This is an important component in the implementation of Role-Based Access Control (RBAC). It blocks all traffic to the server while authenticating the user and protecting sensitive endpoints. As an example, it can be used to make sure that the "Admin Dashboard"

(/admins/dashboard) is not accessible to customers, and, at the same time, to ensure that customers can access their private booking history (/bookings/index) safely.

B. Database Version Control (Migrations Plugin)

The migrations are handled by the cakephp/migrations plugin, which is used to handle the database schema in a programmatic manner. The alterations in the database structure are not created manually as tables but are coded. Rentify has adopted Clean Baseline strategy of migration, in which the whole schema is stored into one initial migration file. This methodology ensures that the database can be reproducible (or "seeded") with a single command on any machine, and errors in deployment are greatly minimized.

C. CakePdf: Automated Document Generation.

One technical need of the position of Business Logic was the creation of downloadable documents. This was integrated with the friendsofcake/cakepdf library. It is a dynamic tool that converts an HTML representation of an invoice into a rigidly structured PDF file. This enables users to get formal records of their transactions which is a feature that mimics the real time enterprise system features.

D. Data Visualization and Interaction Libraries

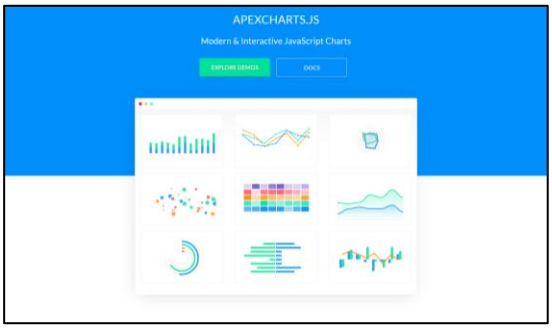


Figure 10: ApexCharts

- **ApexCharts:** ApexCharts are JavaScript software that will be incorporated into the Admin Dashboard to offer visual analytics. They transform raw data, including "Total Revenue" and "Available Cars" into interactive bar charts so that administrators could make decisions based on data at a glance.

Figure 11: Date/Time FlatPickr

- **Flatpickr:** This is a simple JavaScript date picker that is integrated into the booking forms. It communicates with the backend logic to visually gray or disable dates that are already booked or in the past to ensure that users do not make invalid reservations.

Figure 12:FakerPHP

- **FakerPHP:** To show the ability of the system to withstand volume, the fakerphp/faker library is seeded into the database. It is a programmable simulator of realistic dummy data (Malaysian IC numbers, names, and addresses) of more than 50 users and 20 cars so that it is easy to stress test and demonstrate presentations.

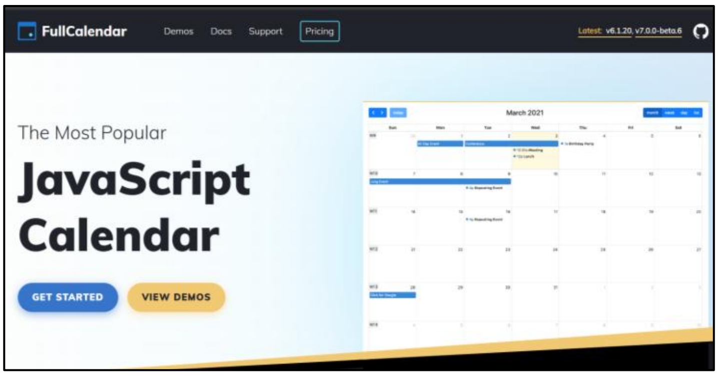


Figure 13: FullCalendar.io

- FullCalendar.io: Installation used to drive the Booking Interactive Calendar module. This library gives a detailed timeline of the view that represents the availability of vehicles and schedules of reservations. It enables administrators to effectively monitor booking times, reservation time on multiple vehicles (e.g., Axia, Saga, Serena) and allows them to operate the daily operations in an easy-to-use interactive grid format.

4.3 Front-End Architecture and Design Strategy

The interface (UI) is used as the main point of contact with all the interactions. In order to achieve a high level of responsiveness and a professional aesthetic, which are directly stated in the project evaluation rubric, the system uses a hybrid design strategy. This style merges the systematic arrangement of the Bootstrap framework and an original custom

Figure 14:BootStrap 5.3

- **Responsive Framework (Bootstrap 5.3.0):** The visual layer of the application is created on the basis of **Bootstrap 5.3** which is a powerful CSS framework that utilizes a mobile-first flexbox grid system. The reason behind this choice is that this framework will make the application be not only fully functional, but also visually consistent across different viewports of devices, be it wide-screen desktop monitors or mobile devices. The development team made sure the layout is fluidly responsive to various screen sizes using the pre-made elements of Bootstrap, including responsive navigation menu, grid containers, and modal windows, to meet the need of having a Professional, responsive and consistent UI design.

- **Design Philosophy (Glassmorphism):** To distinguish Rentify with regular baked framework templates, the interface adopts a Glassmorphism aesthetic. The key features of this contemporary design trend are translucent backgrounds (with CSS properties such as backdrop-filter: blur()) and light border highlights. This style gives the impression of optical depth and hierarchy so that important content, like the vehicle fleet cards or financial statistics, literally floats above the background. This not only adds the modern touch to the application but also helps in increasing user concentration because the interface elements are segregated by depth, as opposed to severe solid lines.

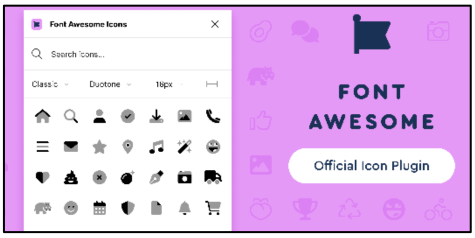


Figure 15:FontAwesome

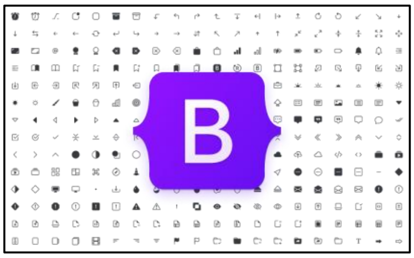


Figure 16:Bootstrap Icons

- **Typography and Iconography:** Legibility and brand identity are achieved via a refined typographic pecking order. The sidebar and the navigation elements are written in **Poppins** (Google Fonts) to give the elements a geometric clarity and clarity at small size. The use of **Raleway** and **PP Neue Montreal** as headings and body text, creates a professional impression of sophistication. **Font Awesome 6.4** and **Bootstrap Icons 1.11** are also incorporated throughout the system to even lessen the cognitive load. The icons are immediate visual indications of the most common actions (a Trash can to delete something or a download icon to create a PDF), which makes the interface easy to use even by a first-time user.

4.4 Client-Side Interactivity and User Experience (UX)

While the CakePHP backend manages data integrity, the frontend user experience is refined through client-side scripting. This layer handles dynamic behavior, allowing for real-time feedback and asynchronous communication without requiring full page reloads.

The CakePHP backend ensures the integrity of data, and the frontend user experience is optimized with the help of client-side scripting. This layer can deal with dynamic behavior and this ensures that real time feedback is possible and that communication can occur asynchronously without having to reload the entire page.

Figure 17: jQuery

- **DOM Manipulation (jQuery 3.7.1):** jQuery is used to make it easier to traverse an HTML Document Object Model (DOM) and to handle events. It is exceptionally helpful in the Booking Engine, where it helps with the dynamic that allows or disallows form fields depending on the choice of the user. An example is when a user switches on the add-on services of a Chauffeur or GPS, which are immediately followed by the recalculation of the field of the Total Price. This real time feedback loop is a necessity of the requirement of the "Payment Simulation", as it will display the correct cost to the user prior to checkout.

Figure 18:SweetAlert

- **Improved Notifications (SweetAlert2):** To make the system more refined than the native, intrusive alert dialogs of the browser, the system incorporates the SweetAlert2 library. This software offers responsive modal popups that are aesthetically pleasing to use in system feedback. It is essential to workflow safety. On the one hand, when an administrator tries to remove a category of cars or abandon a reservation, a SweetAlert modal comes into the picture and asks to confirm the action explicitly ("Are you sure?") or ("This action cannot be undone"). This implementation specifically responds to the need of the rubric to have user friendly validation and error management.

Figure 19: Ajax

- **Asynchronous Data Fetching (AJAX):** To eliminate the conflicts of the bookings, the system uses **AJAX (Asynchronous JavaScript and XML)** requests in the booking interface. Upon selecting a particular vehicle, the frontend will asynchronously request the BookingsController to deliver a list of, for example, disabled date which are those dates when the vehicle is already booked by a different customer. The calendar widget is then given these dates to be visually blocked so that users cannot physically pick invalid date ranges and this will minimize chances of making booking mistakes.

5.0 UI OVERVIEW

The Rentify application has been developed with a user interface that is custom-designed and follows the Glassmorphism design language. The aesthetic decision, which includes the use of transparent components, blurred backgrounds, and faint edges, was made to make the system stand out of the usual boilerplate-based templates and make a user experience feel modern and high-end. Outside of aesthetics, the interface focuses on usability by having a coherent visual hierarchy and responsive interface, which makes the interface work seamlessly on desktop and mobile interfaces.

5.1 Layout & Navigation Design

In order to ensure consistency in navigation, a persistent layout structure is used as specified in the templates/ layout/ default.php (Public) and templates/ layout/admin.php (Private) files. The main navigation code is contained in a custom sidebar component.

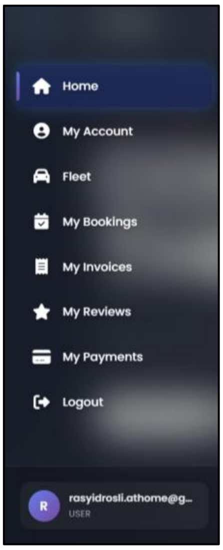


Figure 20: Glasspmorphism Sidebar

- **Glassmorphism Sidebar:** It uses a transparent background (rgba(255, 255, 255, 0.05)) and the CSS backdrop-filter: blur(25px) property in the primary navigation, which is in templates/element/sidebar.php. This gives it the effect of frosted glass through which the background colors can somewhat bleed through without overcrowding the scene. The sidebar also incorporates state management: the page that is in use will automatically be given a faint blue glow with a left-border accent to guide the user throughout the system. Moreover, menu items are dynamically displayed, with the authentication session being used to show or hide them depending on the role of the user (conceal the links in the Manage Fleet menu of regular customers).

Figure 21: Fixed Header

- **Fixed Header & User context:** A fixed header bar (templates/element/header.php) is a fixed block of the viewport that keeps some of the most important utilities at the top of the viewport. It contains the brand identity and a User Dropdown menu to manage their profile and safely log out. The background of the header is given a gradient to ensure that the text is well contrasted and readable against the page content.

5.2 Role-Specific Dashboards

The system uses unique landing interfaces depending on the operational requirements of the various user roles, which ensures a high degree of separation of concerns.

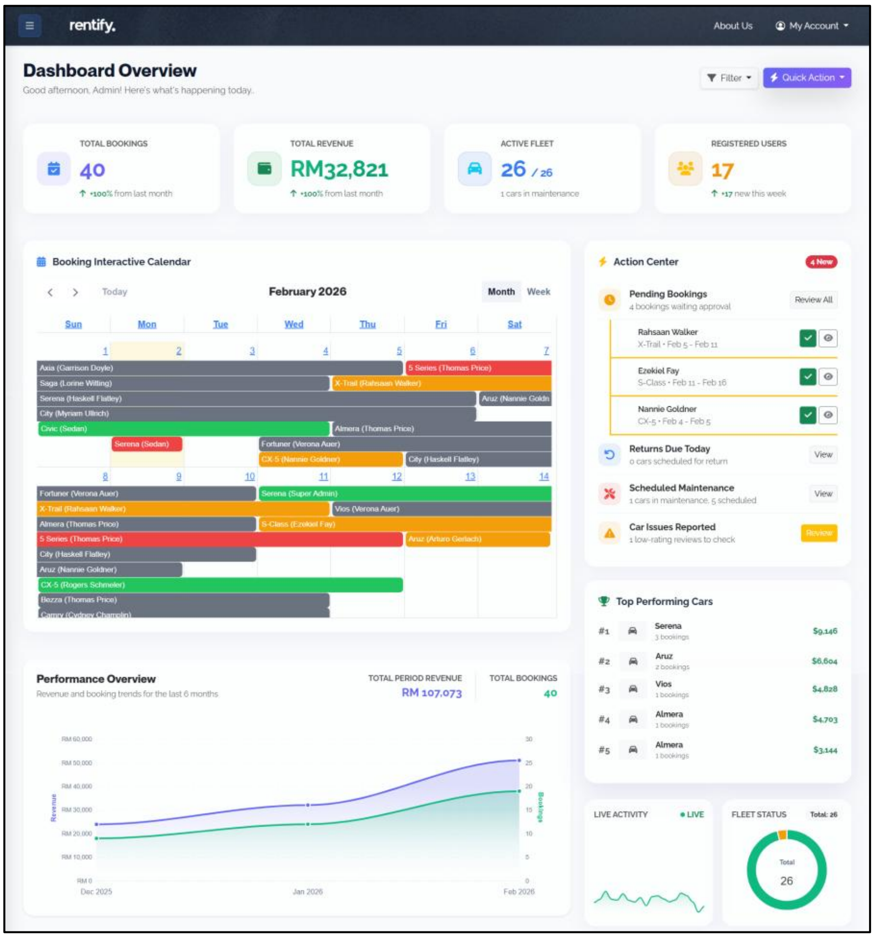


Figure 22: Admin Dashboard

- **Administrative Command Center:** The Admin Dashboard is the main center of business operation. It was developed based on the admin.php layout and it is based on the card-based architecture (Bootstrap Cards) that displays the high-level statistics like the Total Revenue and Active Bookings. This design enables administrators to know the health of the system without having to go through various sub-menus.

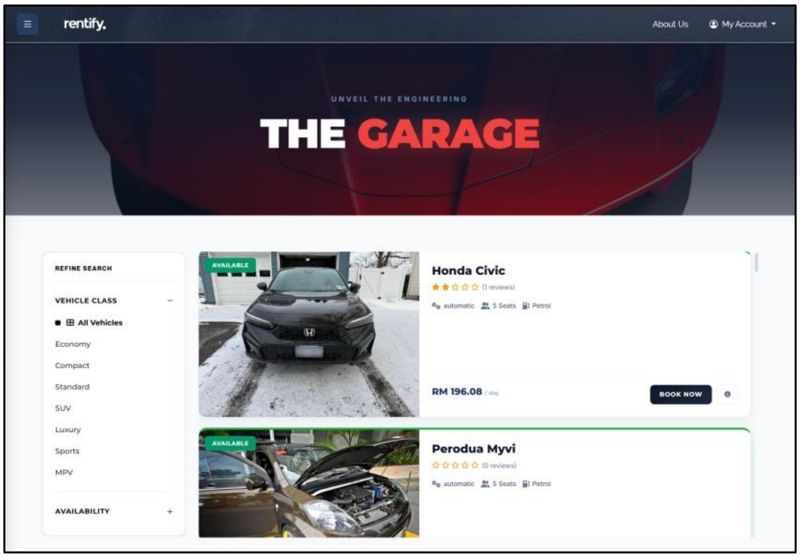


Figure 23: Fleet Gallery

- **Customer Fleet Gallery:** To the end-user, the interface is no longer management-centered but is discovery-centered. The Customer Homepage is a digital showroom and it has a grid system which is responsive to show the vehicle fleet. Each car is displayed as a card with the key decision-making information, like Daily Rate, Type of Transmission and a high-quality thumbnail picture to entice the user to browse and continue with the booking.

5.3 Data Management & CRUD Interfaces

The main component of the Rentify backend is efficient data handling. The system combines DataTables library (datatables-custom.css) to improve the standard HTML tables offered by the framework.

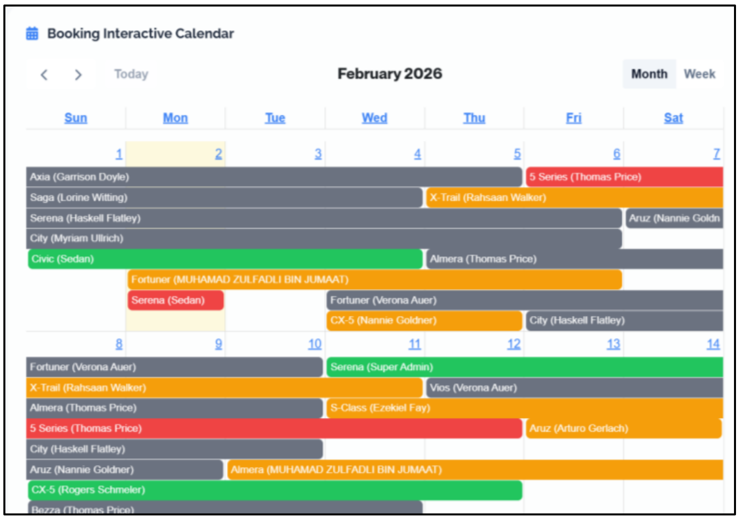


Figure 24: Booking Interactive Calendar

- **Interactive Data Tables:** Complex information-like the Booking History or User Registry, is displayed in sortable paginated tables. This interface enables administrators to handle high amounts of data without straining the eyes.

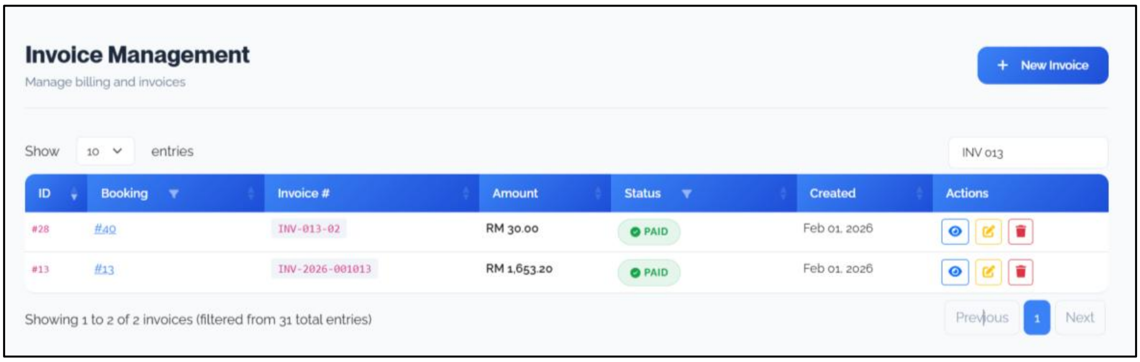


Figure 25: Search & Filtering

- **Search & Filtering:** Each of the table headers carries a real-time, client-side search box. The given feature allows filtering records instantly, for example, finding a booking by an invoice ID or user name without causing a page reload, which makes the administrative process much faster.

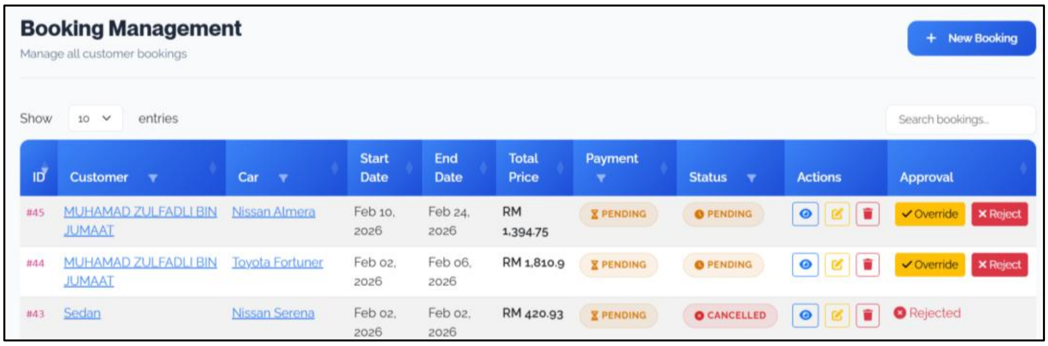


Figure 26: Status Indicator

- **Visual Status Indicators:** Badges (Green, Yellow, etc.) should be used in status fields to enhance scanability (Confirmed, Pending, etc.). The iconic action buttons, Eye (View), Pencil (Edit), and Trash (Delete), are provided on each row to offer an easy access to CRUD functions.

5.4 Forms & Input Validation

Data entry is based on the standardized forms with the help of the CakePHP FormHelper and the Bootstrap 5 styles to guarantee the correctness of the data input and its convenience.

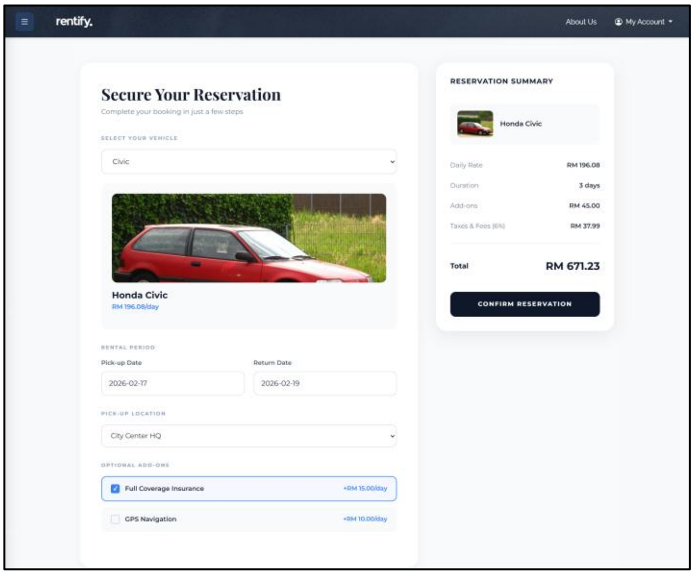


Figure 27: Vehicle Booking Form

- **Input Design/ Usability:** Floating labels and logical field grouping are used in the forms to minimize user error. Complex data types are supported by specialized input widgets; the booking form is using the Flatpickr library to make it easy to select a date in the format of a date picker.

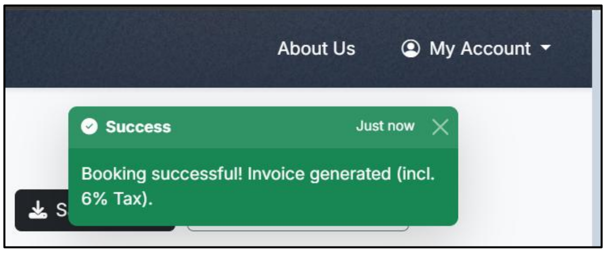


Figure 28: Successful Toast Notification

- **Validation & Feedback:** The validation strategy used by the system is dual. Client-side HTML5 attributes have in-built feedback and server-side validation assures integrity of data. When submitting the form, users are immediately provided with feedback in form of Flash Messages (Toast notifications) which can be green (successfully saved) or red (validation error) and the user will never be left wondering what happened to their action.

5.5 Mobile Responsiveness

Rentify is completely responsive according to the current web standards. The interface uses the media queries and grid system in Bootstrap 5 to scale well with the different screen sizes.

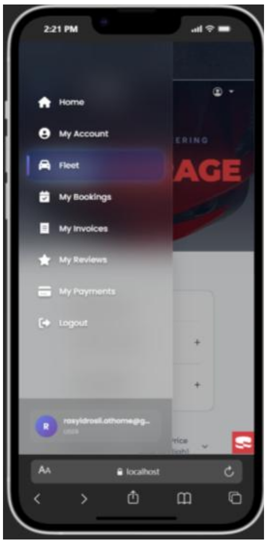


Figure 29: Mobile View

- **Adaptive Navigation (Off-Canvas Pattern):** On mobile gadgets (less than 768px in width) the sidebar is automatically concealed to create a maximum screen space. A “Hamburger Menu” is used in place of it. When this toggle is switched on, the navigation drawer will slide to the left and allow a maximum number of menu items to be displayed without cluttering the mobile display.

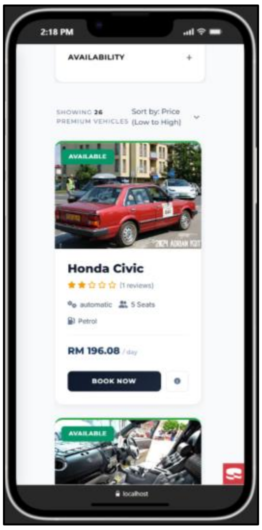


Figure 30: Fleet from Mobile View

- **Fluid Grid Layouts:** Multi-column layouts, like the Car Fleet grid, will collapse to a single column of a stack on mobile devices. This stacking effect grants a content to be readable and usable through vertical scrolling, thus avoiding the possibility of horizontal zooming.

5.6 User Feedback & Interaction

SweetAlert2 is applied to the critical user interactions to avoid operational errors and increase the sense of system robustness.

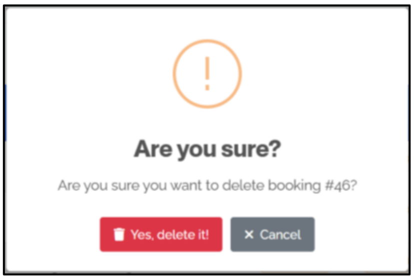


Figure 31: Toast Notification To Delete

- **Destructive Action Protection:** The high-risk actions are substituted with modal dialogs rather than the standard browser alerts. In a situation where one tries to delete a record (a car or category), the system will interrupt the process with a SweetAlert modal that will say, “Are you sure? This cannot be undone." This measure is effective to avoid the loss of the data by mistake, and it corresponds to the best practices of error prevention in interface design.

6.0 FEATURES & FUNCTIONS

6.1 Registration

6.1.1 Client-Side User Registration

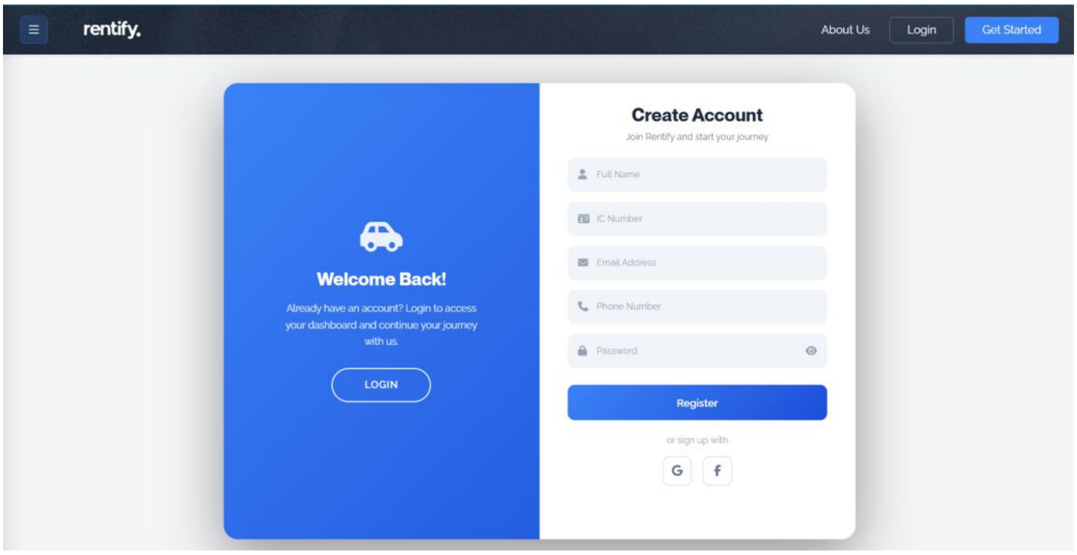


Figure 32: Sign up page for new user

Navigation Buttons

- **Menu & About Us:** These help you find your way around the website and learn more about the service.
- **Get Started / Login:** Quick links to either start a new account or sign into an old one.

The Form (Register)

- **The Boxes:** This is where you enter your personal details like your name, ID, and contact info.
- **Register Button:** Once you've filled everything out, you click this to save your info and officially create your account.
- **Login (on the blue side):** A shortcut to the sign-in page if you accidentally clicked "Register" but already have an account.

Social Links

- **Google & Facebook Icons:** These are dummy buttons. While they look like shortcuts to sign up using your social media, they are currently just placeholders and don't perform any action.

6.1.2 Multi-Role Login Portal

This page is the universal entry point where both **Regular Users** and **Administrators** sign in to access their accounts.

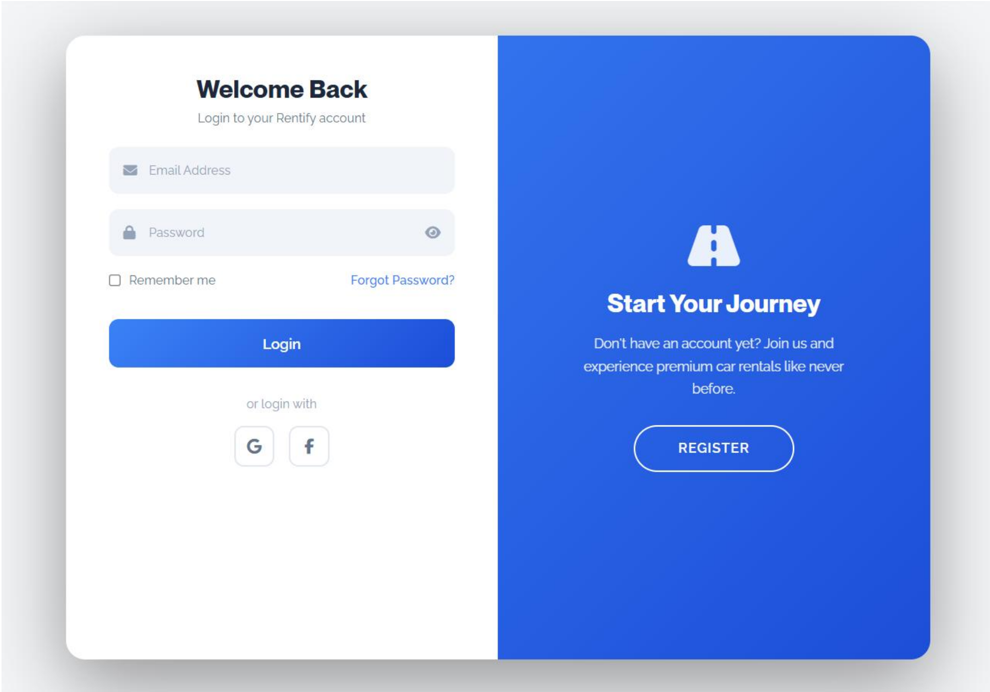


Figure 33 : Log In pages for multi-role user

Login Features

- **Email & Password Fields:** These are the input boxes where you type your login credentials.
- **The Eye Icon**: A utility that lets you see the password you've typed to ensure it is correct.
- **Forgot Password:** A clickable link that helps you recover your account if you can't remember your password.
- **Login Button**: The primary blue button used to submit your information and enter the platform.

Navigation & Placeholders

- **Register Button (Blue Side):** If you are a new user or admin who hasn't created an account yet, this button takes you to the registration page.
- **Remember Me:** This is currently a dummy button. It is a placeholder for a future feature that would save your login details on your browser.

6.2 User Features & Functions

6.2.1 User-Specific Landing Page

This is the main screen a **regular customer** sees after logging in. It is designed to look premium and makes it easy for the user to start browsing for a car.

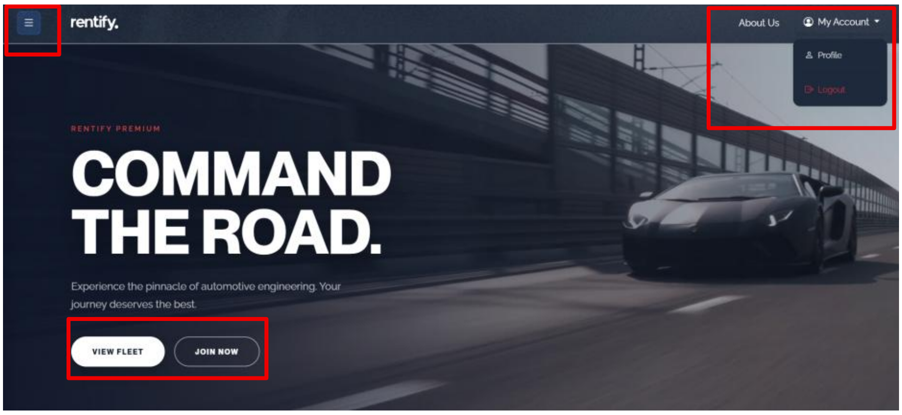


Figure 34: Landing Page for customer

Account & Navigation

- **My Account Dropdown:** This is the most important change after logging in. It confirms you are in your account.
- **Profile:** A button to go to your personal page to see your details or rental history.
- **Logout:** A simple button to safely sign out of your account when you are finished.
- **Menu & About Us:** These stay at the top so you can still find general information about the company.

Main Action Buttons

- **View Fleet:** This is the main goal for a user. Clicking this takes them to a list of all the luxury cars available to rent.
- **Join Now:** An extra button to encourage users to sign up for special memberships or start a booking immediately.

6.2.2 User Sidebar Navigation

When a logged-in **User** clicks the **hamburger menu** on the top-left, this sidebar appears. It acts as the main control panel for managing their rentals and account details.

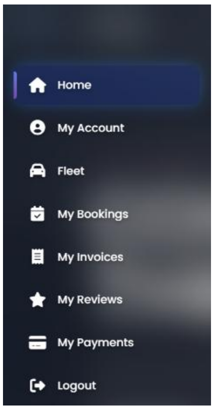


Figure 35 : Sidebar menu for users

Menu Features & Functions

- **Home:** Returns you to the main "Command the Road" dashboard.
- **My Account:** This is where you can view and edit your personal profile information.
- **Fleet:** Takes you to the full catalog of luxury cars available for rent.
- **My Bookings:** Shows a list of your current, upcoming, and past car rentals.
- **My Invoices:** A place to view and download your billing statements and receipts.
- **My Reviews:** Allows you to see the feedback and ratings you have given for previous rentals.
- **My Payments:** Manage your saved credit cards or check your payment history.
- **Logout:** Securely signs you out of the Rentify platform.

6.2.3 My Account (User Profile)

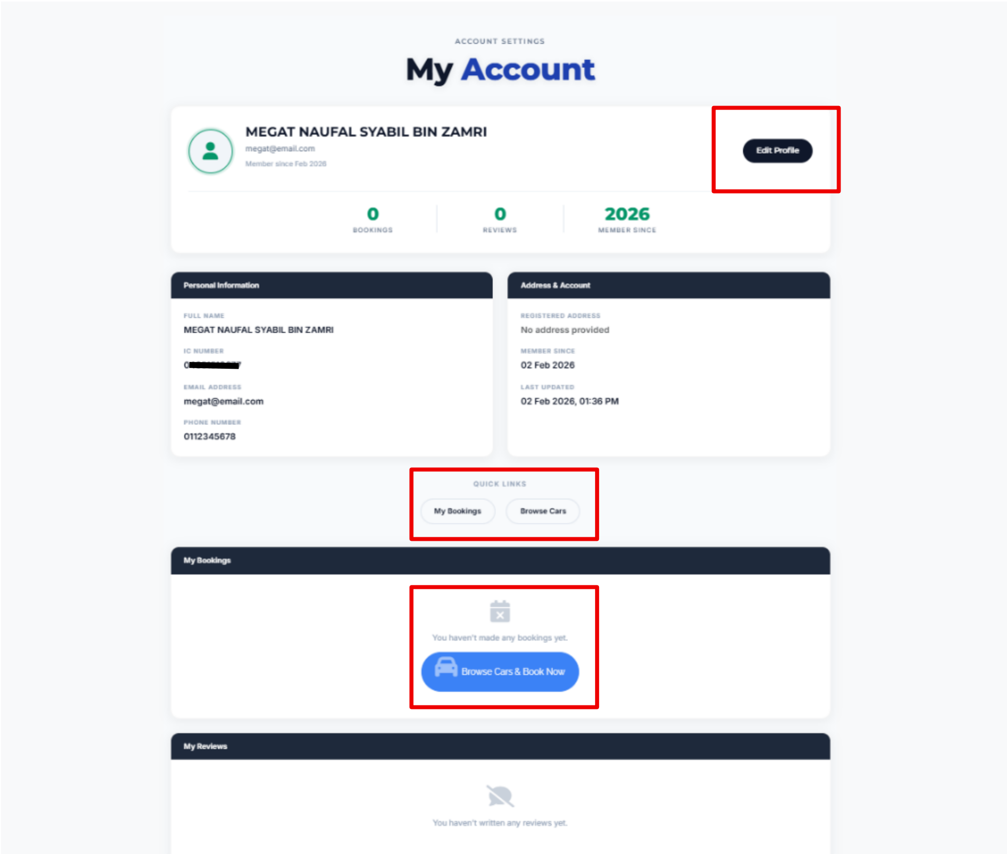


Figure 36 : My account page for users

This page gives the user a complete overview of their activities and personal details.

Account Overview

- **Summary Stats:** Quick counters showing how many Bookings and Reviews the user has made, plus their Member Since year.
- **Personal Information**: A clear display of the user's name, ID (IC Number), email, and phone number.
- **Address & Account:** Shows the registered home address and the exact date and time the account was last updated.
- **Edit Profile Button:** A shortcut at the top to change any of this information.

Quick Navigation & History

- **Quick Links:** Fast buttons to jump straight to My Bookings or Browse Cars.

- **Activity Sections:** Dedicated areas at the bottom to see a history of car bookings and reviews given. If no cars have been rented yet, a "Browse Cars & Book Now" button appears to encourage the user.

6.2.4 Edit Profile Features

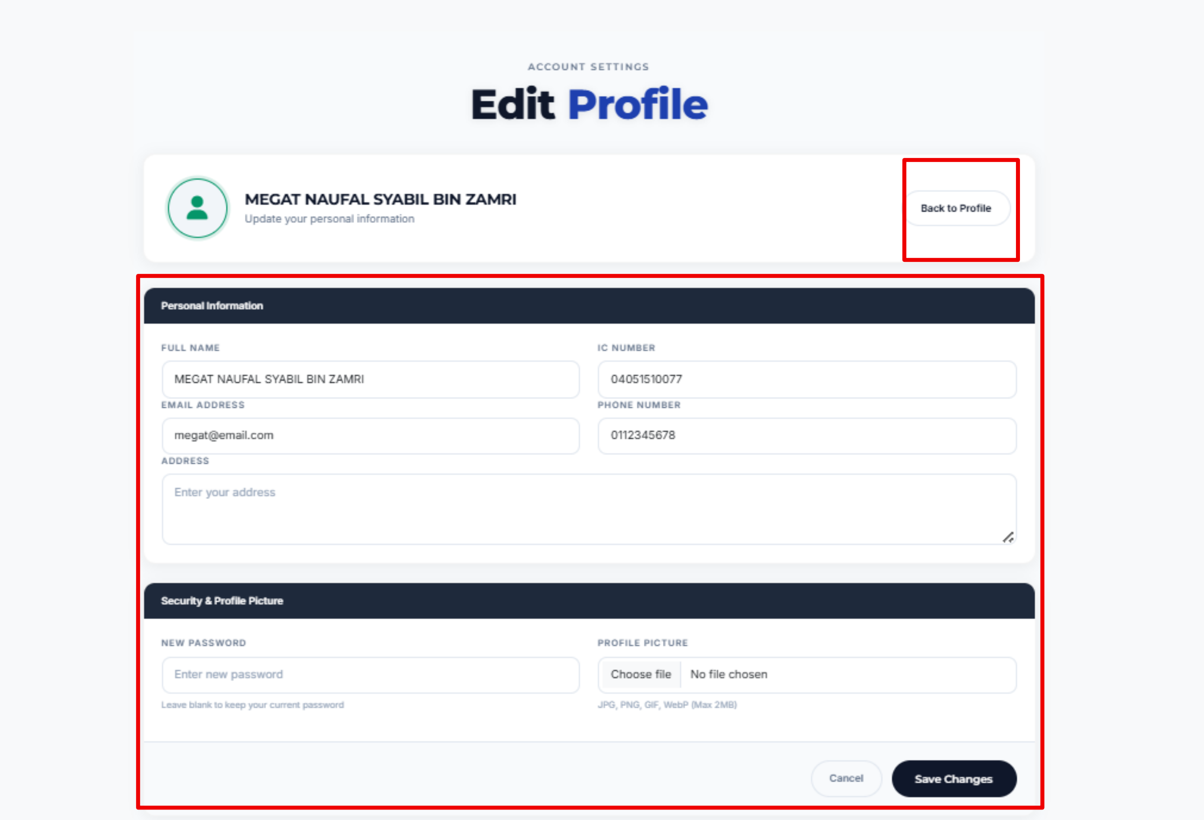


Figure 37 : Edit Profile page for users

This is the page where users can update their information to keep it current.

Updating Details

- **Personal Info Boxes**: Users can type in a new name, phone number, or updated home address.
- **Security (Password)**: A dedicated box to change the account password. If left blank, the current password stays the same.
- **Profile Picture:** A file uploader where users can add a personal photo (supports JPG, PNG, GIF, and WebP formats).

6.2.5 Vehicle Fleet & Search

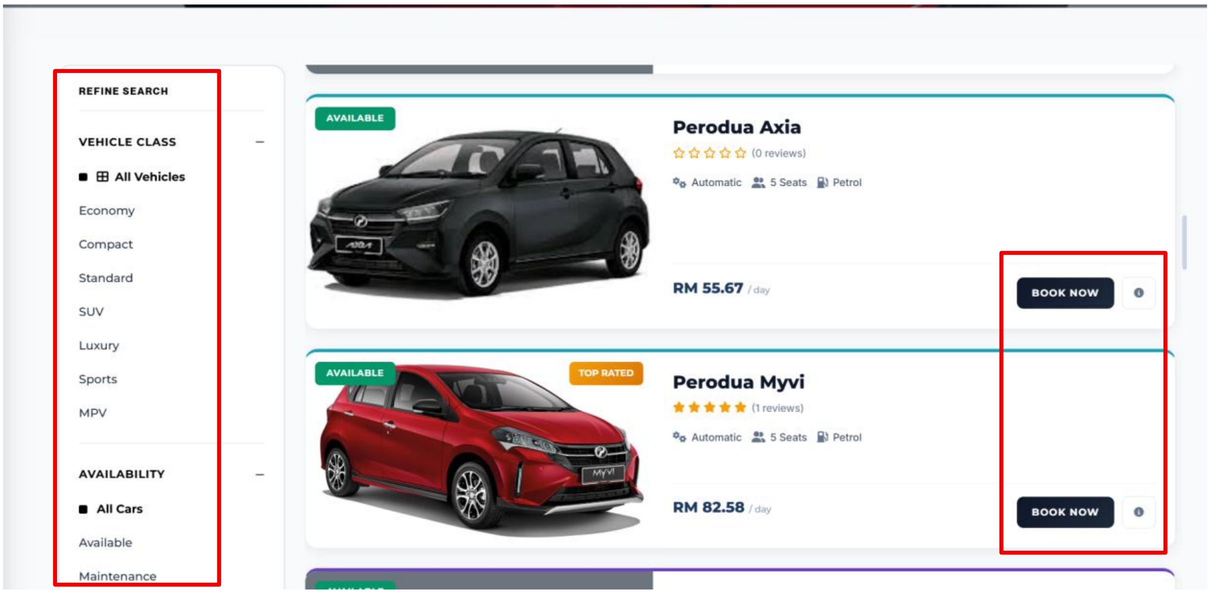


Figure 38 : Fleet pages for users

Buttons used to filter and select cars for rental.

- **Vehicle Class Filters (Economy, SUV, Luxury, etc.)**: Buttons that sort the list to show only specific types of cars.
- **Availability Filters:** Sorts the fleet by "Available" or "Maintenance" status.
- **Book Now:** The primary action button on a car card to start the rental process for a specific vehicle.
- **Info Icon (Small 'i')**: Likely opens a modal or page with more technical details about that specific car.

6.2.6 Bookings Pages

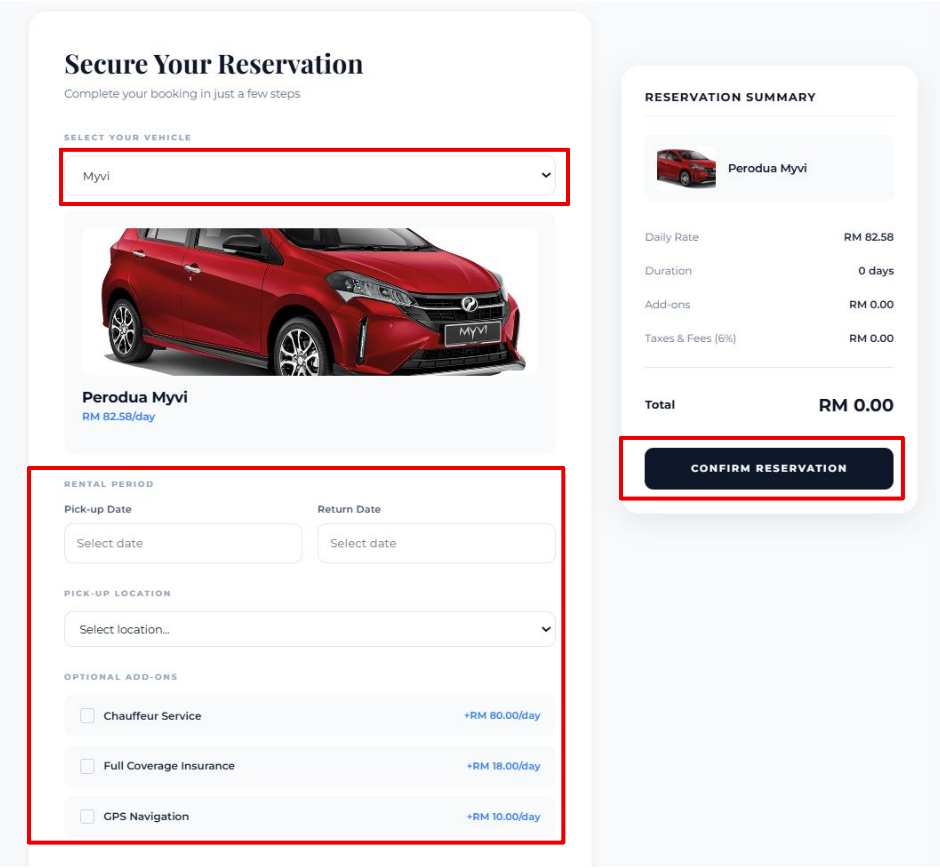


Figure 39 : Bookings Pages

Once a car is selected, this page collects the specific details for the rental.

- **Vehicle Selector**: A dropdown menu that allows you to double-check or change the car model you are booking.
- **Rental Period**:
    - **Pick-up Date:** A button/field to select the day you want to start the rental.
    - **Return Date:** A button/field to select when you will return the vehicle.
- **Pick-up Location**: A dropdown menu to choose which branch or area you will collect the car from.
- **Optional Add-ons**: Checkbox buttons to add extra services like a Chauffeur, Full Coverage Insurance, or GPS Navigation.
- **Reservation Summary:** A side panel that automatically calculates the Daily Rate, Taxes, and Total Price based on your choices.

- **Confirm Reservation Button:** The final button to submit your booking request and secure the car.

6.2.7 Post-Booking Invoice & Payment

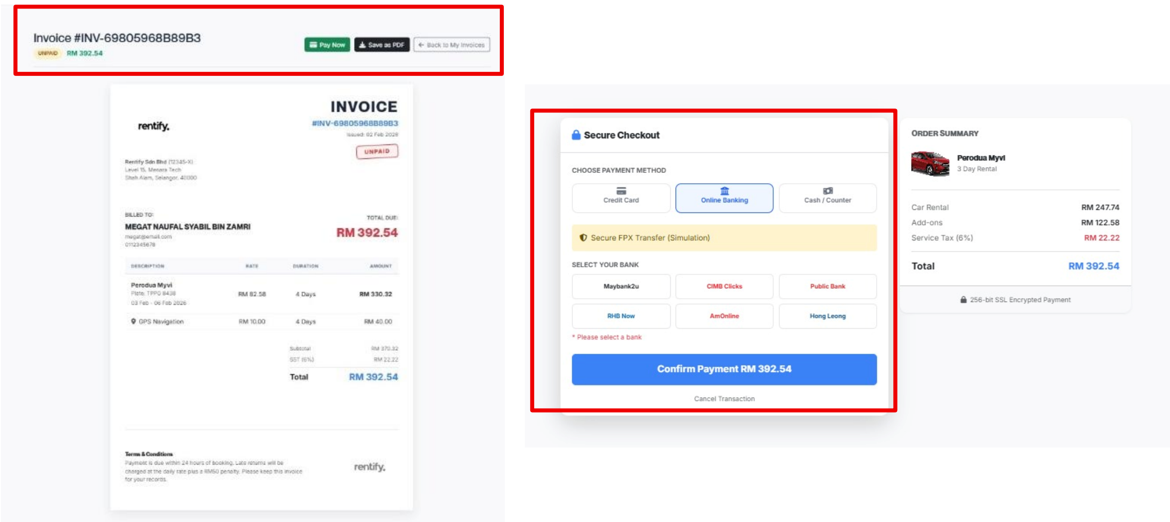


Figure 40: Invoice for rentals system and payments section

Invoice Features

The pop-up invoice serves as a detailed billing summary before the final transaction.

- **Rental Summary:** Lists the specific vehicle details, the rental period (start and end dates), and the pick-up location.
- **Detailed Breakdown:** Shows the daily rental rate, applied taxes (like VAT or service tax), and any selected add-ons (GPS, insurance, or chauffeur).
- **Transparency:** Clearly states the Total Amount Due and may include information on security deposits or holds.
- **Status Indicator:** Initially marked as "Pending" or "Draft" until the payment is processed.

Payment Method Functionalities

The checkout screen allows the user to choose how they want to pay for the rental.

- **Credit/Debit Card**: The primary option for most users. It involves entering card details or selecting a previously saved card for faster checkout.
- **Online Banking / FPX**: Allows direct transfers from a bank account, a common method in regions like Malaysia.
- **Pay at Counter (Optional):** In some setups, this button allows the user to reserve the car online but pay physically when picking it up.

Action Buttons

- **Confirm Payment:** The main button to authorize the transaction and finalize the booking.
- **Download Invoice**: Allows the user to save a PDF copy of the billing details for their records.
- **Back to Booking:** Lets the user return to the previous screen to edit their dates or car choice if they spot an error on the invoice.

6.2.8 My Reservations (Booking Status)

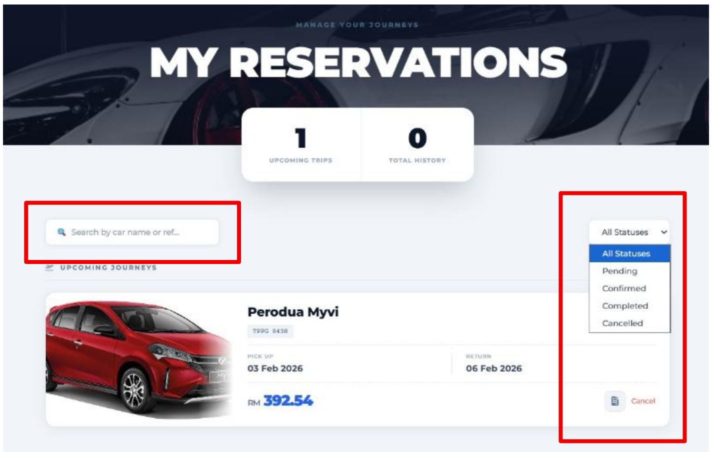


Figure 41: Booking status page for users

After paying, you can manage your trip here.

- **Status Filter Dropdown:** A button to sort your trips by Pending, Confirmed, Completed, or Cancelled.
- **Search Bar:** Lets you find a specific booking by typing the car name or reference number.
- **Cancel Button:** A small button on the trip card used if you need to cancel your bookings.

6.3 Admin Features & Function

6.3.1 Admin Dashboard Landing Page

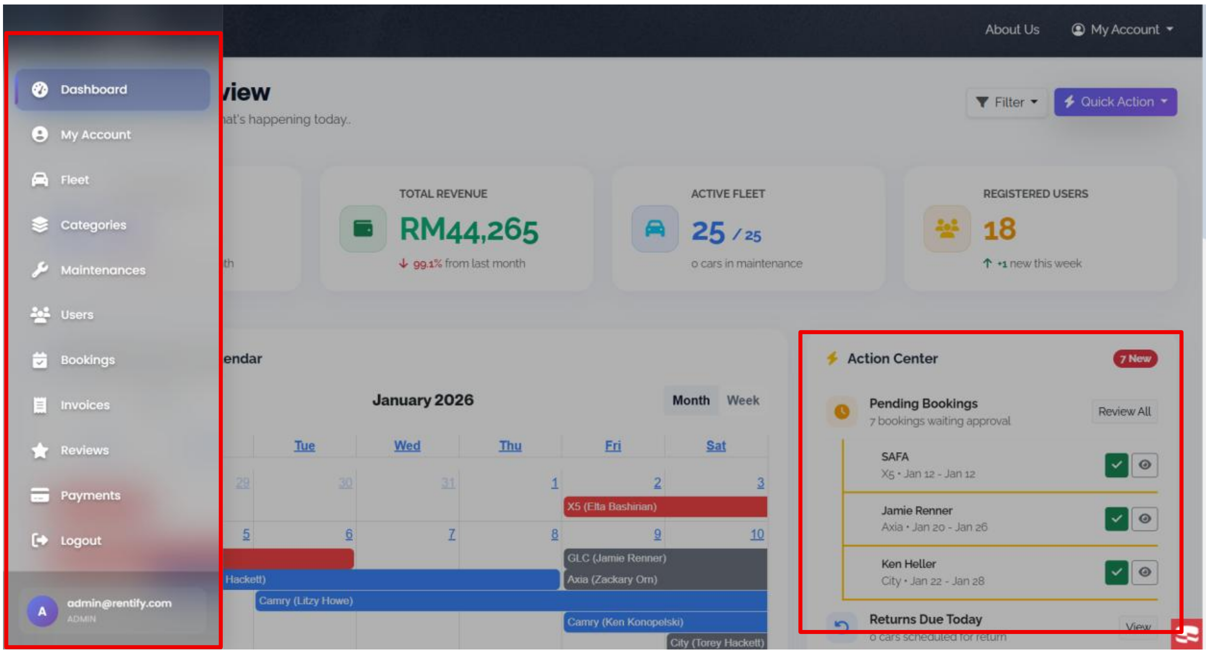


Figure 42 : Admin dasboard and menu

i. Real-Time Data Summary Cards

At the top of the page, key metrics provide an immediate "health check" of the rental business:

- **Total Bookings:** Displays the total number of successful rentals, often showing a percentage comparison to the previous month to track growth.
- **Total Revenue**: A summary of all earnings from rentals, taxes (SST), and add-ons.
- **Active Fleet Status**: Tracks how many vehicles are currently rented out versus those available or in maintenance.
- **Registered Users:** Shows the total customer base and tracks new sign-ups over a specific timeframe.

ii. Fleet Scheduling & Interactive Calendar

This central tool is the heart of daily operations:

- **Visual Scheduling**: A color-coded calendar view shows exactly which cars (e.g., Luxury, SUV, Economy) are booked for specific dates.
- **Date Range Toggles**: Buttons allow the admin to switch between Monthly and Weekly views to plan for high-demand seasons.

- **Availability Tracking:** Automatically updates the calendar when a booking is confirmed to prevent double-booking

iii. **Admin Hamburger Menu**

The Admin Hamburger Menu is a "Control Center" for the entire platform. Unlike the User menu, which is all about renting and personal history, the admin menu is about managing everyone else.

- **Admin Dashboard:** The main button to go back to your "Home" screen with the calendar and total revenue charts.
- **Manage Users:** A button that opens a list of every customer. From here, you can verify their IDs, edit their details, or block users if needed.
- **Manage Fleet:** This is where you control the cars. You can add new cars, change rental prices, or mark a car as "In Maintenance" so users can't book it.
- **Manage Bookings:** Shows every single rental request on the platform. You use this to approve or reject bookings and track which cars are currently out on the road.
- **Manage Invoices:** A financial list where you can see every bill generated. You can check who has paid and who still owes money.
- **Manage Reviews:** Lets you see what customers are saying. You can delete spam reviews or reply to customer feedback.
- **Manage Payments:** A deep look at the money coming in. You can track bank transfers (FPX) and check if payments are successful.

6.3.2 Navigation & Toolbars in Admin Dashboard

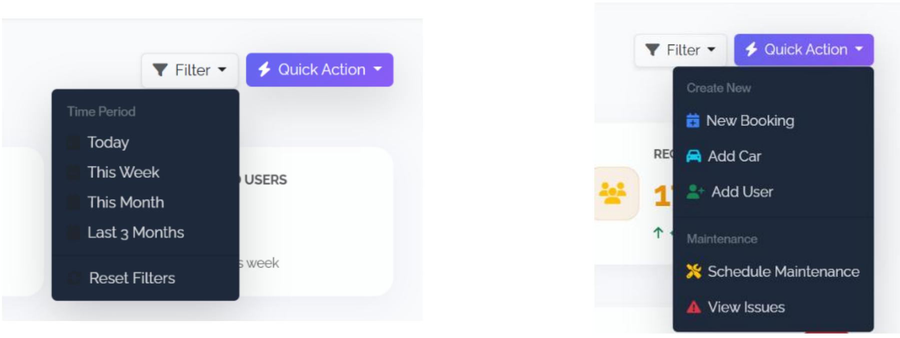


Figure 43 : Filter and quick action button in dashboard for admin page

- **Filter Dropdown:** Allows the Admin to narrow down dashboard data by time periods like **Today**, **This Week**, or **Last 3 Months**.
- **Quick Action (Purple Button):** A "shortcut" button to perform frequent tasks like adding a new car to the system and maintenance.

6.3.3 Car Fleet Management Page

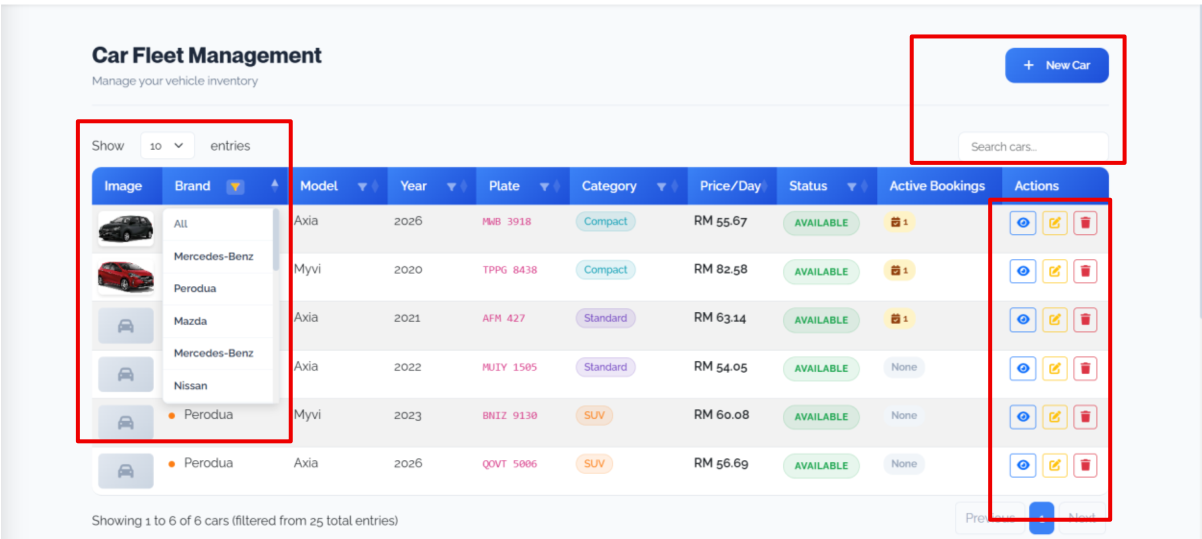


Figure 44 : Car Fleet Management Page for admin

These features allow an Admin to quickly locate specific vehicles within a large inventory:

1. **Fleet Search & Filtering Tools**

- **Search Bar:** A text input field at the top right that allows you to find cars by typing the model's name or license plate number.
- **Column Filtering (Blue Headers):** Each header (Brand, Model, Year, Plate, Category, Status) has small arrow icons. Clicking these allows the admin to:

    - **Filter by Brand:** Select specific manufacturers like **Perodua**, **Mercedes-Benz**, or **Mazda** to narrow the list.
    - **Sort Data:** Organize the fleet by price (lowest to highest), year, or alphabetical order.

- **Entries Dropdown:** A "Show" selector that lets the admin choose how many car records (e.g., 10, 25, 50) to view on one page.

2. Vehicle Management Actions

Located on the far right of every car row, these dedicated buttons provide full control over each vehicle:

- **View Button (Eye Icon):** Allows the admin to see the full technical specifications and high-resolution images of a specific car.
- **Edit Button (Yellow Pencil/Note Icon):** Provides an easy way to **update** car details such as the daily rental price, availability status, or category.
- **Delete Button (Red Trash Icon):** A fast way to **remove** a car from the system if it is no longer part of the fleet.

3. Operational Fleet Status

The table provides a live overview of the current business state:

- **Status Badges:** Clearly marks vehicles as **"AVAILABLE"** in green, making it easy to see which cars can be rented.
- **Active Bookings Column:** Shows exactly how many current rentals each car has (e.g., showing a calendar icon with "1" for a Perodua Axia).
- **New Car Button:** A large blue button at the top right used to add a brand-new vehicle to the inventory database.

4. Navigation & Summary

- **Pagination:** Located at the bottom right, allowing the admin to click "Next" or specific page numbers to browse the full fleet.

6.3.4 Add New Car Page

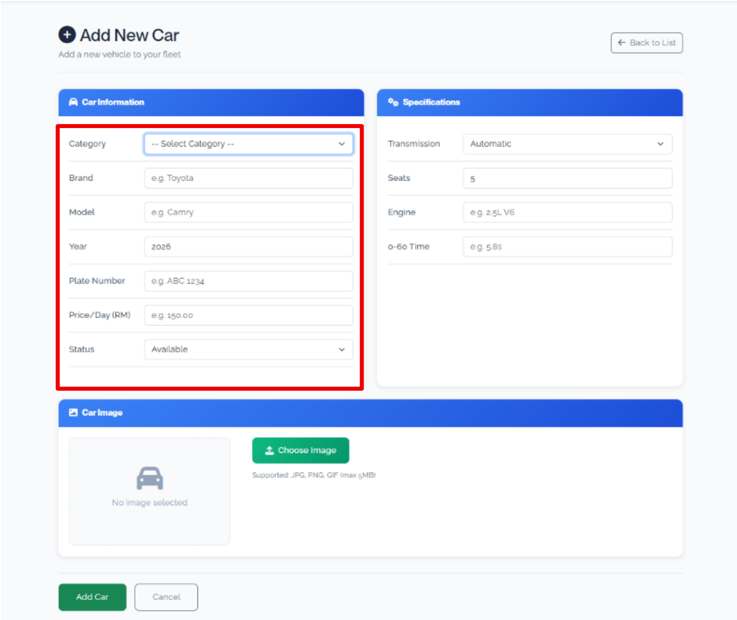


Figure 45 : Add new car page for admin

1. Car Information & Status

This section captures the essential identity of the vehicle:

- **Brand & Model Fields**: Text boxes to enter the manufacturer (e.g., Toyota) and specific model (e.g., Camry).
- **Category Dropdown**: A menu to classify the vehicle, such as **Economy**, **SUV**, or **Luxury**, which helps users filter their search.
- **Year & Plate Number**: Fields to record the manufacturing year and the official license plate for legal and tracking purposes.
- **Price/Day (RM)**: A numerical field where the admin sets the daily rental rate.
- **Status Dropdown**: Allows the admin to set the car's initial state to **Available** or **Maintenance** before it goes live on the site.

2. Technical Specifications

These details provide the high-performance data that premium customers often look for:

- **Transmission**: A dropdown to select between **Automatic** or **Manual**.
- **Seats**: A field to specify the passenger capacity (e.g., 5 seats).

- **Engine & 0-60 Time**: Specific fields for engine size (e.g., 2.5L V6) and acceleration speed to highlight the car's performance.

3. Media & Action Buttons

- **Car Image Uploader**: A dedicated area with a **"Choose Image"** button to upload a professional photo of the vehicle. It supports JPG, PNG, and GIF formats up to 5MB.
- **Add Car (Green Button)**: The final action to save the vehicle to the database and make it part of the live fleet.
- **Cancel / Back to List**: These buttons allow the admin to exit the form without saving any changes.

6.3.5 Booking Management page

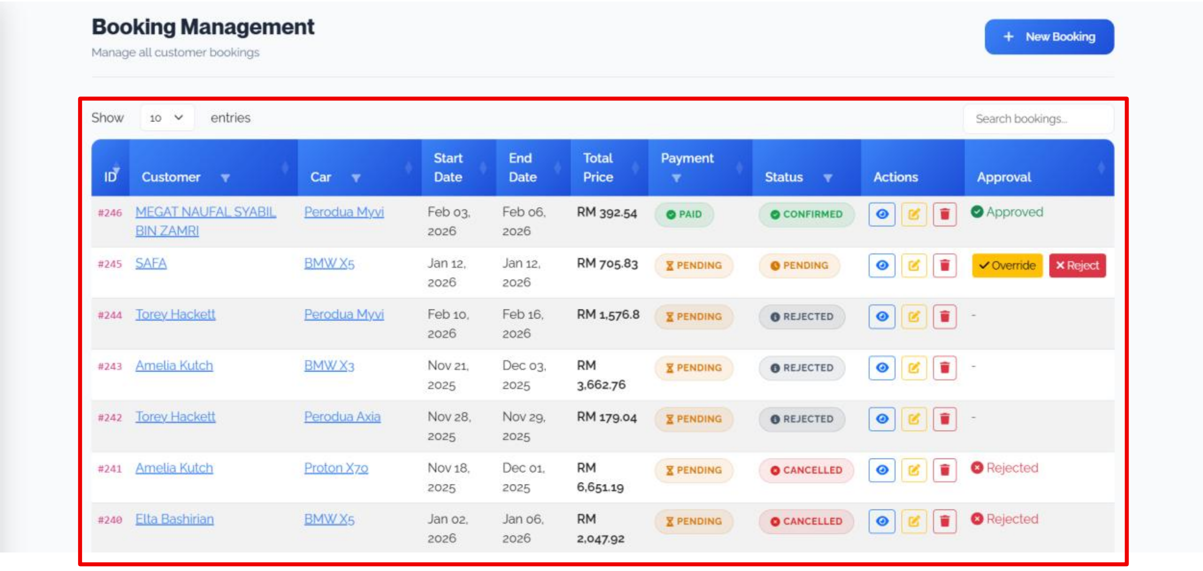


Figure 46 : Booking management page for admin

The **Booking Management** page is the admin's specialized tool for overseeing all customer transactions and controlling the approval workflow.

1. Real-Time Status & Payment Tracking

This table provides a high-level view of every transaction's current stage:

- **Payment Status Tracking**: Displays a **"PAID"** badge in green for completed transactions or a **"PENDING"** badge in orange for those awaiting payment.

- **Booking Lifecycle Labels**: Categorizes each request into clear statuses like **"CONFIRMED"** (green), **"PENDING"** (orange), **"REJECTED"** (grey), or **"CANCELLED"** (red).
- **Approval Workflow Column**: Features dynamic action buttons that change based on the booking status, such as **"Override"** or **"Reject"** for pending requests.

2. Customer & Vehicle Linking

This page acts as a central hub connecting users to specific fleet items:

- **Customer Hyperlinks**: Clicking a customer's name (e.g., **Megat Naufal**) allows the admin to jump directly to that user's specific profile or history.
- **Vehicle Hyperlinks**: Clicking the car model (e.g., **Perodua Myvi**) takes the admin to the technical management page for that specific vehicle.
- **Date Range Overview**: Displays the exact **Start Date** and **End Date** for every booking to help manage fleet availability.

3. Financial Summary per Booking

- **Total Price Column**: Shows the final calculated amount for each rental (e.g., **RM 392.54**), including all taxes and add-ons.
- **Reference IDs**: Assigns a unique **ID number** (e.g., **#246**) to every booking, which matches the customer's invoice and payment history for easy cross-referencing.

4. Global Management Buttons

- **New Booking Button**: A large blue button at the top right that allows admins to manually create a booking (useful for phone-in reservations).
- **Search Bookings**: A dedicated search bar at the top right to find specific records using customer names, IDs, or car models.

7.0 WORKFLOW OF FORM

7.1 CRUD Operational Framework

All the large entities in Rentify, such as Vehicles, Bookings, and User Accounts, follow a standardized lifecycle. This model will make sure that every interaction with the database is valid and consistent throughout the platform.

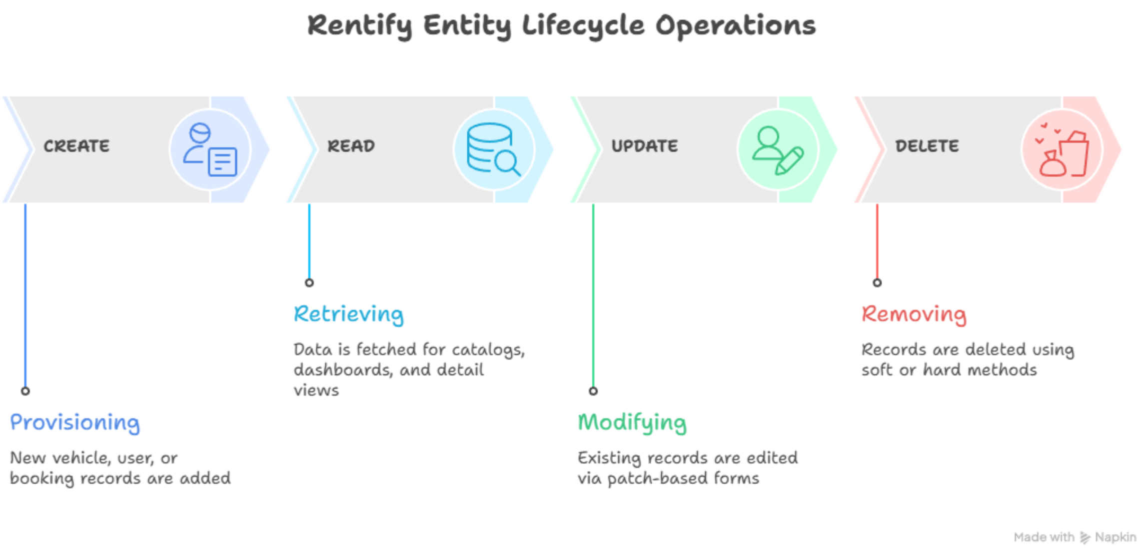


Figure 47 : Rentify entity lifecycle operations

7.2 Workflow Car Management (Administrative)

The vehicle management module is associated with the complex data processing, especially in terms of the alignment of physical properties with digital ones (image uploads).

7.2.1 Data Persistence Logic

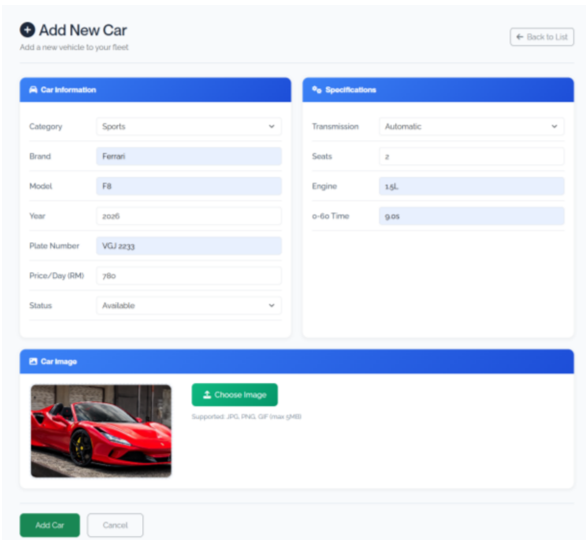


Figure 48 : Data Persistence Logic

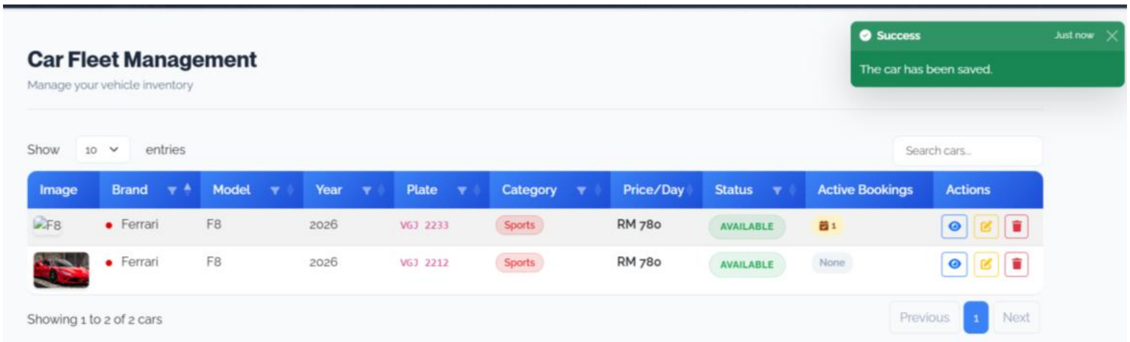


Figure 49: Car fleet management

- **Input Acquisition:** The Administrator adds specifications of the vehicle (Model, Plate Number, Daily Rate) and chooses a high-resolution picture to upload.
- **ImageUploadService:** The POST request is intercepted by the ImageUploadService which verifies the file extensions and dimensions and then stores the asset in the webroot/img/uploads/cars directory.
- **ORM Persistence**: The CarsTable makes use of patchEntity to map the data and file path that has been validated in the database schema.
- **Feedback Loop:** When the save is successful, the system will cause a Flash message and redirect the user to the Inventory Dashboard.

7.3 Booking Lifecycle (Customer and Administrative)

The booking process is the most important business logic of Rentify. It combines real-time availability check, financial calculations and automatic document generation.

7.3.1 Transactional Workflow

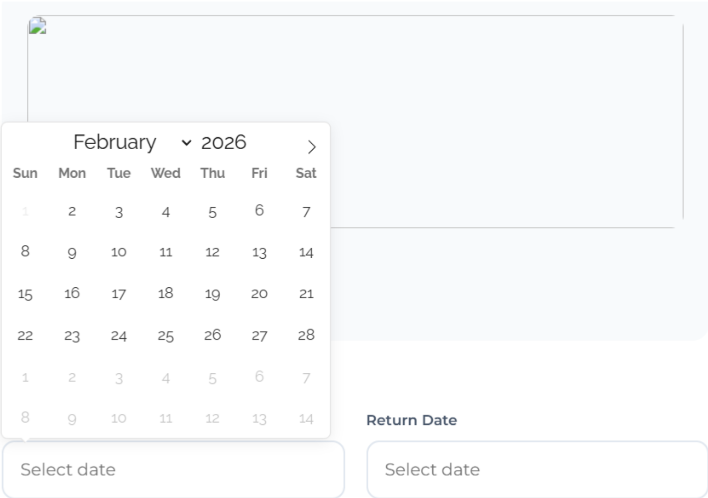
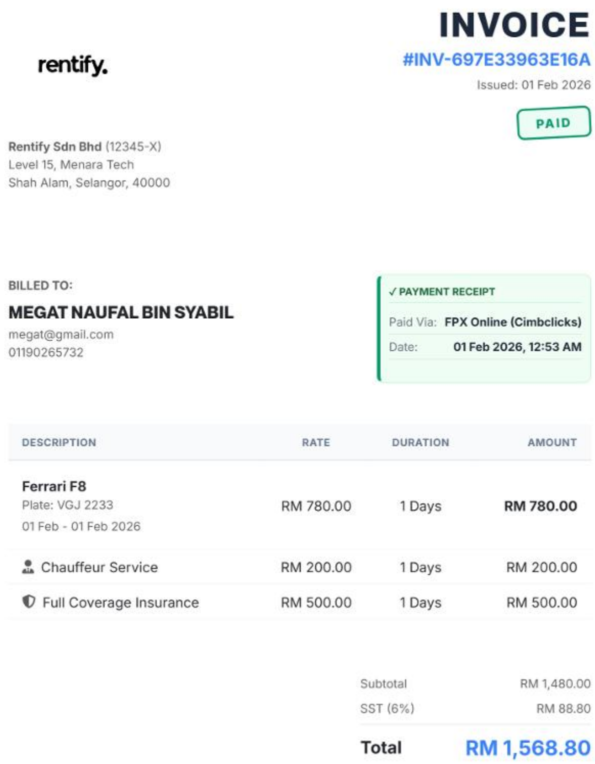


Figure 50: Callender



rentify.

INVOICE
#INV-697E33963E16A
Issued: 01 Feb 2026
PAID

Rentify Sdn Bhd (12345-X)
Level 15, Menara Tech
Shah Alam, Selangor, 40000

BILLED TO:
MEGAT NAUFAL BIN SYABIL
megat@gmail.com
01190265732

✓ PAYMENT RECEIPT
Paid Via: **FPX Online (Cimbclicks)**
Date: **01 Feb 2026, 12:53 AM**

DESCRIPTION	RATE	DURATION	AMOUNT
Ferrari F8 Plate: VGJ 2233 01 Feb - 01 Feb 2026	RM 780.00	1 Days	**RM 780.00**
Chauffeur Service	RM 200.00	1 Days	RM 200.00
Full Coverage Insurance	RM 500.00	1 Days	RM 500.00

Subtotal RM 1,480.00
SST (6%) RM 88.80
Total **RM 1,568.80**

Figure 51: Payment invoices

- **Temporal Selection:** A Flatpickr built-in calendar is used by the customers to choose rental dates. The system sends the information to an API endpoint to turn off occupied or planned maintenance dates.

- **Conflict Validation:** The BookingService runs a specialized ORM query to make sure that there is no overlap. In case a conflict is found, the transaction is terminated with a validation error.
- **Automated Fiscal Calculation:** When validated, the system will automatically compute the 6% SST (Service Tax) and will produce a professional and tax compliant PDF Invoice.
- **Status Synchronization:** After an Administrator confirms the payment (Cash or Online), the system changes the vehicle status and initiates the maintenance - inventory synchronization in case of need.

7.4 Workflow of User Account and Profile.

Rentify has an identity pipeline that handles user transition between public registration and authenticated profile management.

- **Identity Creation:** When a new user is registered, they are automatically given the Customer role which allows them access to the booking engine immediately but denies them administrative capabilities.
- **Profile Mutation:** It allows users to modify sensitive information (Contact details, password) using a patched form that authenticates existing credentials prior to making changes to the database.

7.5 CRUD Security and Data Protection.

To meet the current web security standards and avoid corruption of data, Rentify provides the following protection:

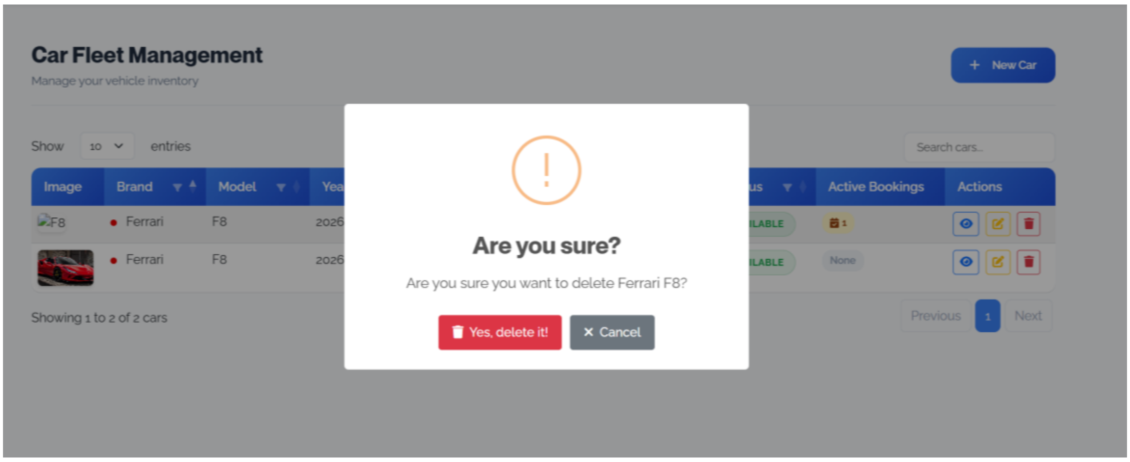


Figure 52 :CRUD Security and Data Protection.

- **Cross-site Request Forgery (CSRF):** Each of them contains a cryptographic token that is hidden to confirm the authenticity of the request origin.
- **Form Tampering Protection:** CakePHP provides the FormProtection component, which functions to lock the fields so that it will avoid malicious injection or alteration of hidden attributes.
- **Destructive Action Protection:** Any DELETE action is encircled with secondary confirmation modals with SweetAlert2 to avoid the loss of records accidentally.

8.0 TEAM ROLES & CONTRIBUTIONS

Figure 53: Team Members

In a bid to deliver the **Rentify** system successfully within 5 weeks duration of the development process required by the **IMS566 course**, the development team embraced a hard and fast domain-driven design. The team understood the intricacy of developing a complete stack application with some of the sophisticated features, such as PDF generation and RBAC security, which led it to a strategic division of the workload into four separate areas of technical interest: **Frontend** (Views), **Business Logic** (Transactions), **Admin Operations** (Management), and **System Integration** (Security).

This **Divide** and **Conquer** approach were critical as it barely allowed the range of each developer, giving each member certain *Controllers* and Views so that someone did not make overlapping edits. The team was able to reduce the number of conflicts related to merging and maximize the output in terms of code efficiency through specialization due to following the "Golden Rules" of the project (using feature branches "*git checkout -b feature/name*" and no direct online pushes to the main branch).

8.1 Role Overview Table

The table below will outline the specific roles, domains and the main responsibilities of each team member:

Member	Name	Role	Domain Focus	Primary Responsibility
Member A	Muhamad Zulfadli	Frontend & UX Designer	Views (The "Look")	The design of the so-called Shop Window (Landing Page, Catalog) and a professional User Interface (UI) design.
Member B	Muhammad Amirul Rasyid	Business Logic Engineer	Transactions (The "Money")	The administration of the Cash Register (Payments), reservation of calculations, and creation of PDF invoices.
Member C	Muhammad Haziq	Admin Operations Specialist	Management (The "Staff")	Construction of the Back Office Dashboard, fleet management and financial KPIs monitoring.
Member D	Safa Muhammad Raziq	System Integrator	Security & Core Logic	As the "Guardian" (Security) performing Authentication, Search Logic and Data Integrity.

Table 2: Table of Roles and Contribution

8.2 Detailed Contributions & Responsibilities

8.2.1 Muhamad Zulfadli (Member A)

Role: Frontend & UX Designer

Domain Focus: User Experience and Views

Muhamad Zulfadli was the **Decorator** of the application, who did the visual presentation and responsiveness. He aimed at gaining a **High Distinction** grade of the interface through application of modern design principles.

- **Glassmorphism & UI Design:**
    - **Landing Page** (Pages/home) was designed, including a Hero Banner and a Featured Cars section in order to make a solid first impression.
    - Adopted a **Glassmorphism** inspired user interface on the admin dashboard to have a high-efficiency, modern user interface.
- **Car Catalog Interface:**
    - Replaced boring tables with a **grid card** of Cars/index list done in a Cars/index list format (Image on top, details below).
    - Created the frontend front-end HTML of the **Search Interface** and **Filter Dropdowns** (Price or Category), which is linked to backend query logic.
- **Visual Enhancements:**
    - Added **FontAwesome** Icons to show visually vehicle specifications (on Seats, on Transmission) rather than simple text.
    - Used **Bootstrap 5** as grid layouts and responsive and Sweetalert2 / Toastr as an easy way to display an error / success popup.
- **Invoice Styling:**
    - Prepared the template of Invoices/view to look like a real **"Paper Styling"** receipt with clear borders and status badges (Green when Paid, Red when Unpaid).

8.2.2 Muhammad Amirul Rasyid (Member B)

Role: Business Logic Engineer

Domain Focus: Transactions & Financial Logic

Muhammad Amirul Rasyid was the **"Banker"** and the **"Mathematician"** performing the complicated calculations and workflow activations of each transaction.

- **Booking Engine Logic:**
  - Wrote the pricing algorithm in *BookingsController* to compute the total price:(Start Date - End Date) Price Per Day.
  - Added logic of **Overlap Prevention** to make sure that no car is booked twice at the same period of time.
  - Added **Flatpickr (JS)** to visually gray-out prior dates in the booking calendar and inhibit invalid dates.
- **Auto-Expire Invoices:**
  - Designed the new **Auto-Expire** feature of the system whereby the system automatically voids any unpaid invoices in case the booking start date has already passed as that would not allow users to pay on an already expired slot.
  - Developed the **Auto-Invoicing** trigger, which creates an in-depth invoice line (retrieving User and Car data) right after a booking has been stored.
- **Payment Simulation:**
  - Developed the Dummy Credit Card form in *PaymentsController* which checks against certain dummy payments (4242-4242) to make it look like a successful payment.
  - Imposed rigid card number validation (16 digits) and CVV (3 digits) validation.
- **Document Export:**
  - Set the engine of **CakePdf** in such a way that it could print tax compliant PDF invoices that can be downloaded by users as a record.

8.2.3 Muhammad Haziq (Member C)

Role: Admin Operations Specialist

Domain Focus: Management and Analytics

Muhammad Haziq was in charge of the Back Office, which developed the Intelligence Admin Dashboard, where the business central command center is located.

- **Intelligence Administration Dashboard:**
  - Created the *AdminsController*, which computes and shows the most important financial KPIs, such as the "Total Bookings, the Available Cars, and the Total Revenue.
  - Added **ApexCharts** that allow seeing fleet availability and revenue growth (Month-over-Month) on the dashboard.
- **Smart Fleet Protection:**
  - Introduced the "Maintenance Logic" which means that cars that are affected by the scheduled maintenance are automatically out of the booking engine or made inaccessible to avoid conflicts in availability.
  - Created a **Maintenances** CRUD interface to record the costs of repairs and monitor the status of vehicles.
- **Operational Workflow:**
  - Created the Approve/ Reject workflow of bookings, which gives admins an opportunity to make a bookings confirmation or cancelation manually, through the master list.
  - Added **DataTable** to administrative index pages to allow instant search, sorting and pagination of large user and booking datasets.

8.2.4 Safa Muhammad Raziq (Member D)

Role: System Integrator & Security Specialist

Domain Focus: Data and Security

Safa Muhammad Raziq was the so-called **Architect** who made sure that the system was safe, the integrity of the database was guaranteed, and the search capabilities worked properly.

- **Security & RBAC:**
  - Added **Authentication Plugin** and added Role-Based Access Control **(RBAC)** to have a strict separation between "Admin" and "Customer" portals.
  - Added *AppController* logic, which redirects unauthorized users (customers attempting to open the dashboard) to the homepage.
- **Advanced Search Engine:**
  - Coded the *CarsController* the backend query logic to drive the frontend search bar allowing a complex querying like WHERE car_model LIKE %Toyota% AND category_id = 2.
- **Database Integrity & Seeding:**
  - Manually fixed the *belongsTo* association in *CarsTable.php* to ensure cars were correctly linked to their categories, resolving a known CakePHP "baking" issue.
  - Utilized **FakerPHP** to seed the database with over 50 realistic Malaysian users and bookings (including IC numbers and addresses) to prepare the system for the final presentation.
- **Stress Testing:**
  - Conducted **"Human Swarm"** stress testing to simulate concurrent bookings and ensure the validation logic held up under load.

9.0 CONTACT INFORMATION

To access technical support, system documentation queries or bug reports relating to the **Rentify Car Rental Management System**, please consult the development team list below.

9.1 Development Team Directory

Name	Role	Responsibility	Student ID / Email
Muhamad Zulfadli	Frontend & UX Designer	UI/UX, Views, Interface Design	2025157497 2025157497@student.uitm.edu.my
Muhammad Amirul Rasyid	Business Logic Engineer	Booking & Payment Transactions	2025160577 2025160577@student.uitm.edu.my
Muhammad Haziq	Admin Operations Specialist	Dashboard & Fleet Management	2025198985 2025198985@student.uitm.edu.my
Safa Muhammad Raziq	System Integrator	Security, Auth & Data Integrity	2025516833 2025516833@student.uitm.edu.my

Table 3: Team Directory

9.2 Technical Resources

- **Official GitHub Repository:** To view the entire source code, commit history, as well as technical documentation, visit the project repository: https://github.com/haziqariff703/rentify

9.3 Support Channels

- **Bug Reports:** Bug reports and any technical problems should be pointed out through the Issues tab in the GitHub repository.
- **Documentation:** The list of the installation instructions and database migrations (*bin/cake migrations migrate*) can be found in the **README.md** file, which is found in the root directory of the project.

10.0 CONCLUSION & REFLECTION

10.1 Overall Conclusion

The **Rentify** Car Rental Management System is an important technical and operational milestone, with its key aspect being the successful implementation of the advanced concepts of web development to solve the business inefficiencies in real life. The project addresses and goes beyond the main requirements of the course, **IMS566**, which required to not only computerize a manual process based on the form but also to develop a full-fledged, collaborative, and database-driven web application. Through the replacement of the archaic manual record-keeping process with a centralized system based on the digital ecosystem, Rentify aborts or removes typical operational threats like duplication of data, unintentional double-bookings, and ineffective tracking of the fleet.

The system is based on a strong technological foundation based on the usage of the **CakePHP 5.x** framework and the use of **PHP 8.1+** and guarantees the system high performance, security, and scalability. The development team provided a solution that goes far beyond the common CRUD (Create, Read, Update, Delete) and includes a database migration system with a Clean Baseline and uses advanced libraries like **CakePdf** to generate documents and **FakerPHP** to seed data with realistic data.

We have created an engine that is able to make intelligent decisions based on complex business logic. The major innovations are the so-called **Smart Fleet Protection** system that corresponds to the automatic combination of the booking engine and maintenance schedules to conceal the cars that are not available and the so-called **Auto-Expire Invoices** that is used to automatically cancel the unpaid bookings when the rental period starts to keep the financial data correct. A dual-interface design (a responsive frontend to make a reservation) based on **Bootstrap 5** and a **Glassmorphism inspired** Intelligence Admin Dashboard is also applied, increasing the user experience. It is a command center with high efficiency as it operates on **ApexCharts** to display key financial KPIs and fleet availability in real-time.

Finally, Rentify shows that the team is skillful in the implementation of the entire **Systems Development Life Cycle (SDLC)**. Since it is necessary to create a normalized Entity Relationship Diagram (ERD) and implement a secure **Role Based Access Control (RBAC)** system that will strictly separate Administrator and Customer rights, each stage of development was carefully planned and implemented. What comes out of the coursework is not only a satisfactory completion of assignment but a solid, high-quality prototype that can transform the business of small-to-medium car rental companies.

10.2 Project Reflection

The 5 weeks development experience of Rentify offered the team with profound technical knowledge and experience of working in teams on software engineering. There are a number of challenges that we have faced and needed to innovate to overcome them.

1. **Difficulties in technology that have been overcome:**

    - **Framework Mastery (CakePHP):** It was difficult to switch to a rigorous framework of Model View Controller (MVC). In particular, the adoption of the **CakePdf** engine to create invoices was reported to be notoriously challenging as explained in the developer handbook. The team was required to carefully configure the plugin in such a way that the HTML views could be properly displayed as downloadable and taxable PDF files to customers.

    - **Database Integrity (The "Category Fix"):** In the course of creating the search logic, we had found a significant problem that the typical Bake command does not properly tie the Cars table and the *CarCategories* one. This necessitated a manual addition in *CarsTable.php* so as to write the *belongsTo* association so that the car catalog would show Sedan as opposed to Category ID: 1.

    - **Complex Logic Implementation:** The application of the so-called logic of bookings, namely the **Overlap Prevention**, was more complicated than expected. We were required to consider Sandwich dates on which a new booking may commence before but end after a current booking. The resolution of this made possible the elimination of any vehicle being accidentally double booked.

2. **Key Lessons Learned:**

    - **The Significance of “Stress Testing":** We got to know that code that works on a single user would sometimes go wrong as the load increases. Using the Human Swarm method of testing and the **FakerPHP** to populate the database with more than 50 realistic user entries (including Malaysian IC numbers), we managed to find and resolve the validation errors before the actual presentation.

- **Domain Driven Collaboration:** To prevent the experience of the so called **Merge Conflicts** and code overwrite, we strictly followed the rules of our handbook, which is known as the Golden Rules. Naming of certain Controllers (Member B to *BookingsController*, Member C to *Adminscontroller*) helped us learn the importance of a modular development and strict communication when working in a team setting.

- **NYP with Libraries:** This is because we understood that a good backend should have a great frontend. The use of external libraries such as **Flatpickr** as the calendar (to turn the past dates grey) and **Toastr** as the notification system enhanced the user experience greatly and made the system seem professional and not just utilitarian.

10.3 Future Enhancements

Although Rentify is a complex system, we have pointed out three main areas that we can focus on in further development to match the platform to the commercial standards:

- **Live Payment Gateway Integration:** The current system has a dummy controller with Payment Simulation. One of the future iterations would include the introduction of a live Malaysian payment gateway API, like **ToyyibPay** or **Stripe** to effect actual credit card and FPX transactions safely.

- **Mobile Application Development:** To make such access more achievable, a special mobile app (created in React Native or Flutter) might be created. This would enable the customers to control their My Bookings dashboard simultaneously and possibly open cars with a keyless entry system right through their smartphones.

AI-Based Fleet Analytics: The existing dashboard is powered by **ApexCharts** to show past data. Our suggestion is to add an AI-based predictive analytics module, which employs booking history to predict demand surges (in the case of a holiday), and reply to it by optimizing its pricing of rental.

11.0 REFERENCES

Chmln. (n.d.). *Flatpickr*. Introduction. https://flatpickr.js.org/examples/#inline-calendar

contributors, phpMyAdmin. (n.d.). *PhpMyAdmin*. https://www.phpmyadmin.net/

FAKERPHP / Faker. FakerPHP / Faker. (n.d.). https://fakerphp.org/

Font awesome. Font Awesome. (n.d.). https://fontawesome.com/

GitHub · change is constant. github keeps you ahead. GitHub. (n.d.). https://github.com/

Laragon. (n.d.). *Getting started*. https://laragon.org/docs

Mark Otto, J. T. (n.d.). *Bootstrap*. Bootstrap · The most popular HTML, CSS, and JS library in the world. https://getbootstrap.com/

The most popularjavascriptcalendar. FullCalendar. (n.d.). https://fullcalendar.io/

Open source javascript charts for your website. ApexCharts.js. (2020, March 7). https://apexcharts.com/

openjsf.org, O. F.-. (n.d.). JQuery. https://jquery.com/

SWEETALERT2. SweetAlert2. (n.d.). https://sweetalert2.github.io/